Republic of Tunisia

Ministry of High Education & Scientific Research

University of Carthage

Higher School of Communication of Tunis (SUP'COM)

Option:

Telecom **S**ystems **A**rchitectures (AST)

Graduation Project report

FMCW Radar Target Classification for Intelligent Traffic Monitoring

---

March, 1th to August, 31th 2019

Achieved by: **Imen BEN AMARA**

3rd year Engineering Student

Supervised by: **Pr. Adel GHAZEL**

Professor in Telecommunications at SUPCOM

Ing. Hykel TKITEK

Design Engineer at WEVIOO

Host Company: **W**evioo France

Academic year: 2018-2019

Signatures

Signature & Stamp

Pr. Adel GHAZEL

Signature & Stamp

Ing. Hykel TKITEK

Acknowledgments

This project implementation would not have been possible if I did not have a support of many individuals. Therefore, I would like to extend my sincere gratitude to all of them.

First of all, I would like to express my sincere and deepest gratitude to my academic supervisor Mr **Adel Ghazel**, Professor at SUP'COM, for his guidance, valuable suggestions, and constant assistance throughout this internship.Secondary,I 'm thankful to Mr **Hykel Tkitek**, Team Builder at Wevioo Group,for his fruitful collaboration and supervision during my internship.Last but not least, I would like to thank Mr **Emir El Aroui**, Embedded Systems Skills Center Manager at Wevioo Group, for the collaboration and the help provided to me during my internship.

Nevertheless, I express my gratitude toward Wevioo team engineers for their help and kind cooperation, which help me in completion of this project.

Abstract & Keywords

Road traffic has a significant effect on metropolitan activities, especially during peak hours when it impacts on areas such as the economy and the environment. During the past few years, new technologies in the fields of information and communication have led to the introduction of intelligent transportation systems.

The objective of our project, based on the host company application requirements, is to develop, implement and test an algorithm solution for the 24 GHz FMCW Radar targets classification. Within this objective, the conducted project activities started by studying mostly used artificial intelligence based techniques for the classification of moving road traffic targets, including vehicles, bike and pedestrians. By considering both Machine Learning (ML) and Deep Learning (DL) approaches, we carried out field traffic data collection by using the DemoRad 24 GHz Radar Kit from Analog Devices than proceeded to data pre-processing to build the training and test data base. Different training models have been developed for ML and DL techniques and their performances have been tested on real time field traffic scenarios. A classification accuracy of 92% have been obtained with SVM model, 99% with ANN and 75% with CNN-based model.

Key words: FMCW Radar, Road Traffic Monitoring, Radar Targets Classification, Machine Learning, Deep Learning, Features engineering.

Résumé

Le trafic routier occupe une place majeure sur les activités métropolitaines, en particulier pendant les heures de pointe, lorsqu'il affecte des domaines tels que l'économie et l'environnement.Ainsi, Les nouvelles technologies de l'information et de la communication ont permis, en l'espace de quelques années, de mettre en œuvre des systémes de transport intelligents.

L'objectif de notre projet , basé sur les exigences de l'entreprise hote , est de développer, mettre en oeuvre et tester une solution d'algorithme pour la classification des cibles radar FMCW à 24 GHz. Dans le cadre de cet objectif, les activités , menées dans le cadre du projet , ont débuté par l'étude de techniques principalement basées sur l'intelligence artificielle pour la classification des cibles en mouvement, notamment des véhicules, des cyclistes et des piétons.

En prenant en compte les approches Machine Learning (ML) et Deep Learning (DL), nous avons collécté , en premier lieu , des données sur le trafic sur terrain , en utilisant le kit de radar DemoRad 24 GHz d'Analog Devices, puis procédé au prétraitement des données , pour constituer la base de données de formation et de test. Différents modéles de formation ont été développés pour les techniques ML et DL , et leurs performances ont été testées sur des scénarios de trafic sur le terrain en temps réel. Une précision de classification de 92% a été obtenue avec le modèle SVM, 99% avec ANN, 75% avec CNN.

Mots-clés: FMCW Radar, Surveillance du Trafic Routier, Classification des Cibles Radar, Apprentissage Automatique(ML), Apprentissage Approfondi(DL), Analyse des Descripteurs.

Acronyms

AI: Artificial Intelligence.

ANN : Artificial Neural Networks.

AR: Accuracy Rate.

AUC : Area Under Curve.

BNB : Bernoulli Naive Bayes.

BOM : Bill of Material.

CNN : Convolutional Neural Networks.

CW: Continuous Waves.

CM: Confusion Matrix.

DL : Deep Learning.

DNN : Deep Neural Networks.

DTC : Decision Tree Classifier.

LoR : Logistic Regression.

FFT : Fast Fourier transform.

KNN : k-Nearest Neighbors.

FM : Frequency Modulation.

ML : Machine Learing.

MIMO: Multiple Input Multiple Output.

NB: Naive Bayes.

SVM : Support Vector Machine.

PC : Principal Component.

PCA : Principal Component Analysis.

RD : Range Doppler.

ROC : Receiver Operating Characteristic.

RFR : Random Forest Regression.

2D-CNN : 2 Dimensional Convolutional Neural Networks.

Contents

Signature i

Acknowledgments ii

Abstract & Keywords iii

Résumé iv

Acronyms v

Table of Contents ix

List of Figures xii

List of Tables xiii

General Introduction 1

1 Framework of the Internship Project 3

1.1 Introduction . . . 3

1.2 Internship Project Overview . . . 4

1.2.1 Target Recognition for Traffic Monitoring Applications . . . 4

1.2.2 Target Recognition Techniques . . . 5

1.2.3 Application Target Recognition Requirements . . . 5

1.3 Project Objectives and Design Methodology . . . 7

1.3.1 Design and Development Activities Objectives . . . 7

1.3.2 Design and Validation Methodologies . . . 8

1.3.3 Project Planning . . . 9

1.4 Internship Project Work Environment . . . 10

1.4.1 Hosting Company Presentation . . . 10

1.4.2 Radar Hardware Platform . . . 11
1.4.3 Hardware & software Tools . . . 12
1.5 Conclusion . . . 13

2 Radar Target Recognition Design Considerations 14

2.1 Introduction . . . 14
2.2 Survey on AI-based Techniques for Radar Target Classification . . . 15
2.2.1 Radar Target Classification Processing Stages . . . 15
2.2.2 Machine Learning based Techniques . . . 16
2.2.3 Deep Learning based Techniques . . . 20
2.3 Radar Sensor Platform Configuration . . . 21
2.3.1 DemoRad 24 GHz Radar Transceiver Configuration . . . 21
2.3.2 DemoRad 24 GHz Radar Target Detection Architecture . . . 24
2.3.3 DemoRad 24 GHz Radar Target Detection Data Format . . . 25
2.4 Design Methodology for the Radar Target Classification Solution . . . 26
2.4.1 AI-based Radar Target Classification Architecture . . . 26
2.4.2 Data Set Building . . . 28
2.4.3 LearningClassification Model Building and Evaluation . . . 31
2.5 Conclusion . . . 34

3 Field Test Campaign and Data Set Analysis 35

3.1 Introduction . . . 35
3.2 Field Test Scenarios Definition . . . 36
3.2.1 Data Collection Environment . . . 36
3.2.2 Data Collection . . . 37
3.3 Data Set Building . . . 41
3.3.1 Data Set Cleaning . . . 41
3.3.2 Data Set Format . . . 46
3.3.3 Target Classes Data Set Analysis . . . 47
3.4 Data Set Pre-processing . . . 49
3.4.1 Data Set Pre-processing Objectives . . . 49
3.4.2 Features Definition . . . 50
3.4.3 Features Selection . . . 52

3.5 Conclusion 56

4 Radar Target Classification Solution Implementation 57

4.1 Introduction 57

4.2 Machine Learning-based Classification Solution 58

4.2.1 ML-based Models Development 58

4.2.2 ML-based Models Accuracy Analysis 59

4.2.3 SVM-based Model Optimization 60

4.3 Deep Learning-based Classification Solution 64

4.3.1 CNN-based Model Development 64

4.3.2 ANN-based Model Development 68

4.4 Field Test Results of Target Classification Solutions 70

4.4.1 Classification Results for Field Test Scenario 1 70

4.4.2 Classification Results for Field Test Scenario 2 72

4.4.3 Classification Results for Field Test Scenario 3 73

4.4.4 Classification Results for field Test Scenario 4 75

4.5 Conclusion 77

General Conclusion 78

Appendix A 80

Appendix B 87

Appendix C 88

Appendix D 89

References 92

List of Figures

1.1 Example of Radar Target Recognition for Traffic Monitoring Applications . . . 4
1.2 WEVIOO footprint around the world . . . 10
1.3 Block Diagram of the $24GHz$ DemoRad Multichannel Radar Sensor . . . 12
1.4 DemoRad Kit . . . 12

2.1 Radar Target Recognition Processing . . . 15
2.2 Machine Learning based Radar Target Classification Processing Stages . . . 16
2.3 Deep Learning based Radar Target Classification Processing Stages . . . 16
2.4 Decision Tree Classifier Concept . . . 17
2.5 k-Nearest Neighbors Concept . . . 18
2.6 Logistic Regression Concept . . . 18
2.7 Support Vector Classifier Concept . . . 19
2.8 Principal Component Analysis Concept . . . 20
2.9 ANN Architecture with one hidden layer . . . 20
2.10 Architecture of CNN . . . 21
2.11 DemoRad Radar Transceiver Components . . . 22
2.12 DemoRad Transceiver Configuration Software . . . 23
2.13 Target Detection Processing Stages Architecture . . . 25
2.14 Time-domain representation of a single chirp measurement . . . 26
2.15 Radar Target Classification Project Work-flow . . . 27
2.16 Radar Target Recognition Processing using ML . . . 27
2.17 Radar Target Recognition Processing using DL . . . 28
2.18 Training Data Base ML Construction . . . 28
2.19 Training Data Base DL Construction . . . 29
2.20 Feature extraction Concept . . . 30
2.21 Feature Selection Concept . . . 31
2.22 ROC Curve With High AUC . . . 32

3.1 Mobile data collection platform . . . 36
3.2 Measurement Set-up for Controlled Environments . . . 37
3.3 Measurement Set-up for Uncontrolled Environments . . . 37
3.4 Sequence length for Pedestrians Class . . . 41
3.5 Range Doppler Map for Pedestrian Class Measured Dataset . . . 41
3.6 Reflected Power Repartition of Radar Frames before and after Cleaning Processing for Pedestrians Class . . . 42
3.7 Sequence length for Bike Class . . . 43
3.8 Range Doppler Map for Bike Class Measured Dataset . . . 43
3.9 Reflected Power Repartition of Radar Frames before and after Cleaning Processing for Bike Class . . . 44
3.10 Sequence length for Vehicle Class . . . 45
3.11 Range Doppler Map for Vehicle Class Measured Dataset . . . 45
3.12 Reflected Power Repartition of Radar Frames before and after Cleaning Processing for Vehicle Class . . . 46
3.13 Repartition of Useful Frames after Data Cleaning for the three Classes and ML & DL Techniques . . . 46
3.14 Video recording labeled with real time . . . 47
3.15 Database distribution for ML and DL Techniques . . . 47
3.16 Radar Target Detection Signal Processing Stages . . . 49
3.17 Features extracted histogram . . . 52
3.18 Database Features Scores Results with SelectKBest . . . 53
3.19 Database Features Scores Results using the Extra Tree Classier . . . 53
3.20 Correlation heatmap for the ML Dataset . . . 54
3.21 Features Selection Results with RFE Algorithm . . . 55
3.22 PCA Resut . . . 56

4.1 Data Splitting for both ML and DL Datasets . . . 58
4.2 Model Accuracy for ML-based Models . . . 60
4.3 Model Accuracy for the 3 Designed SVM Models . . . 61
4.4 Classification Report for the 3 Designed SVM Models . . . 61
4.5 Confusion Matrix for the 3 Designed SVM Models . . . 62
4.6 AUC-ROC Curves for the Designed SVM Models . . . 63
4.7 CNN-based Classification Processing Stages . . . 64

4.8 CNN-2D Model Design Parameters 66
4.9 Training and Validation Accuracy of the CNN-based Model 67
4.10 Training and Validation Loss of the CNN-based Model 67
4.11 ANN-based Classification Processing Stages 68
4.12 Normalized Confusion Matrix for ANN model 69
4.13 Confusion Matrix without Normalization for ANN model 69
4.14 Classification report for ANN 70
4.15 Classification Test Scenario Result for Vehicle Target Class 71
4.16 Classification Test Result for Pedestrian Target Class 72
4.17 Classification Test Result for Bike Target Class 74
4.18 Classification Test Results for Multi-targets 75
A1 The support vector machine visualized in a two-dimensional feature space . . . 80
A2 Illustration of soft-margin SVM 85

List of Tables

2.1 DemoRad Radar hardware elements 22
2.2 Radar Chirp parameters 23

3.1 Test Scenarios for Pedestrians Target Class 38
3.2 Test Scenarios for Bike Target Class 38
3.3 Test Scenarios for Vehicle Target Class 39
3.4 Measurement Information for controlled environment 39
3.5 Measurement Information for uncontrolled environment 40
3.6 Repartition of Useful Sequences for Pedestrians Class 42
3.7 Repartition of Useful Sequences for Bike Class 44
3.8 Repartition of Useful Sequences for Vehicle Class 45
3.9 Input and Output Signals Definition for the Radar Target Detection Processing Stages 50
3.10 Results of Features Extracted 51

4.1 Results of SVM Model Optimization 60
4.2 Test Scenario Definition for Vehicle Target Class 70
4.3 Classification Accuracy Results for Test Scenario 1 71
4.4 Test Scenarios Definition for Pedestrian Target Class 72
4.5 Classification Accuracy Results for Test Scenario 2 73
4.6 Test Scenario Definition for Bike Target Class 73
4.7 Classification Accuracy Results for Test Scenario 3 74
4.8 Test Scenarios Definition for Multi-targets 75
4.9 Classification Accuracy Results for Test Scenario 4 76

General Introduction

The automatic processing of real-time traffic information is an important feature to analyze the traffic. The number of traffic sensors is quickly growing as well as the amount of useful information available for traffic monitoring applications. As we are moving towards Smart Cities, traffic monitoring becomes an important topic to address in order to improve the security of the road and to maintain good supervision.

At present scenario, most vision-based monitoring techniques requires traffic data from large sensory network which is extremely expensive. Hence, a new traffic monitoring system that is based on analyze and assess traffic situations in real time under virtually all weather conditions is needed where one important part is moving target detection and classification.

On another hand, Radar sensors offer in general the capability to measure extremely accurately target range, radial velocity, and azimuth angle for all objects inside the observation area. These target parameters can be measured simultaneously even in multiple target situations, which is widely used with the new technologies of FM-CW radar, but the development of Artificial Intelligence-based target classification solutions still in prototyping phases in research labs and no mature products are available in the market.

Giving the highly economic potential for Low power FMCW Radar based smart sensors for traffic monitoring and surveillance applications, WEVIOO group is engaged within World leading companies for RD activities related to this topic. It is in this perspective that I had the chance to be selected by WEVIOO Group for an internship in their RD labs in Paris to work on data science field for a period of 6 months started from March to August, 2019.

The main objective of my project is to purpose an intelligent solution for classification road traffic target in urban area using two approaches, the first based on machine learning algorithms and features engineering and the second based on deep learning algorithms. then, we evaluate two model to choose the most performing approach to implement on hardware target.

This report is divided in the following chapters.In The first chapter we will give a brief overview of the host company Wevioo Group, the project context and its objectives besides a definition of the framework and the development tools used.

In the second chapter, we will provide survey on Artificial Intelligence (IA)-based techniques for radar target classification,Design methodology for developing AI-based solution for Radar target recognition.

In the third chapter, we will explain in detail the hole processing from data gathering and cleaning the data until we get ML dataset and DL dataset.

Last but not least, we will present the design of the proposed solution.We will also explain step by step the experiment setting starting from the data collection stage up to the ne-tuning phase.Then,we will analyze and interpret the results of the experiment.

Finally, we conclude with a general conclusion that presents a summary of the work done and opens some perspectives.

CHAPTER 1

Framework of the Internship Project

1.1 Introduction

In this chapter, we aim to give a global overview about the framework of the internship project. Firstly, an overview of the project is going to be presented then the project objectives and Design Methodology is going to be introduced. At the end, a presentation of internship project work environment is going to take place.

1.2 Internship Project Overview

1.2.1 Target Recognition for Traffic Monitoring Applications

Intelligent transportation systems (ITS) are fully involved in the road traffic Monitoring through the dynamic means and equipment used by the government on road and highway networks. They are also major asset for the information of users and more generally for the operation of the road.

Road-traffic monitoring involves the collection of data describing the characteristics of targets and their movement through road networks. targets counts, targets speed, path, flow rates, vehicle length, class (vehicle , bike , pedestrians) are all examples of useful data. Such data may be used for one of four purposes:

- Law enforcement: Speeding vehicles, dangerous driving, illegal use of bike lanes.
- Congestion & Incident detection: Traffic queues, accidents and bike are potentially hazardous to approaching vehicles. If such incidents can be detected then variable message signs and speed limits can be set up-stream in order to warn approaching drivers.
- Increasing road capacity: Increasing the capacity of existing roads is an attractive alternative to building new roads. Given sufficient information about the status of a road network it is possible to automatically route traffic along the least congested roads at a controlled speed in order to optimize the overall capacity of the network.

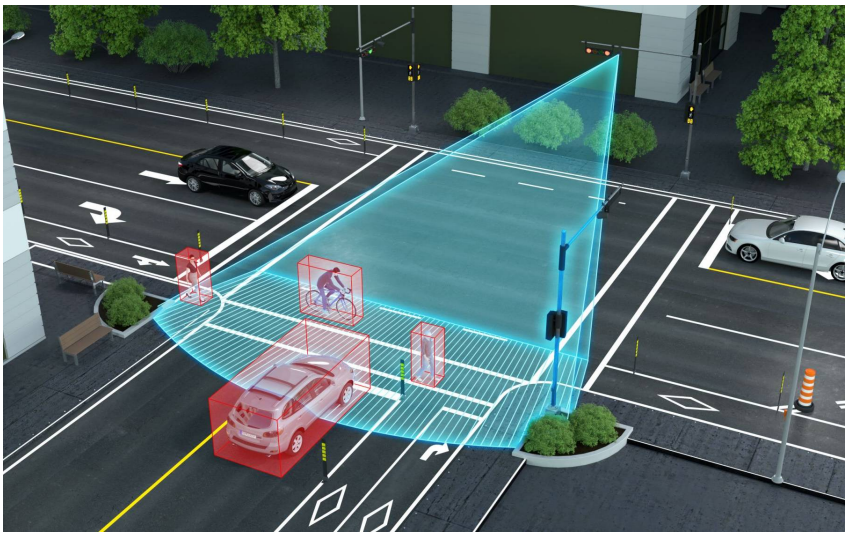


Figure 1.1: Example of Radar Target Recognition for Traffic Monitoring Applications

1.2.2 Target Recognition Techniques

Classification of moving objects based on sensor data is not a new field. In fact, there are several techniques for target recognition in urban areas using different technologies. However, the classification process is usually done at a high level, combining input data gathered from several sensors such as LIDAR, stereo cameras and radars.

The use of multiple sensors and fusion of sensor data can provide a very accurate classification, but also requires heavy processing power. Additionally, the use of multiple sensors will lead to a higher cost for the sensor platform, and the moving parts that are found in LIDAR systems can lead to a reduced hardware robustness of the system.

As one of the sensor-based methods, radar-based moving target recognition has drawn much attention due to the following reasons. Firstly, radar is robust to light and weather conditions, so it is able to be applied in harsh environments. Secondly, radar could protect visual privacy. Instead of capturing the visual shape of the target, the returned signals modulated by the target carry abundant time–varying range and velocity information of targets . Lastly, radar is able to detect target through walls, which makes radar-based moving target recognition applicable to more scenarios. Consequently, radar has been adopted more and more for recognizing moving target for traffic monitoring, recently.

Other methods of target recognition based on radar exist input rely on a deeper knowledge about the radar signal characteristics in order to construct statistical models. Whereas these methods require radar data in its raw form, the radar sensors used in this project deliver heavily processed data in the form of detection points. When dealing with processed radar data in this form, little research has been done before.

The classification system concerned in this project, should not require big amounts of data as input, the calculations to be made should be of low complexity, and it should be executable in a real-time context.

1.2.3 Application Target Recognition Requirements

The following requirements have been defined by collecting data from the host company and our survey on deployed commercial products in the market.

Real time : The system shall detect and classify objects in a real time environment with a predictable execution time. This execution time must be fast enough so that the output can be computed before the next sample.

System output : the class output needs to have a certain degree of accuracy. A condence output (with some sort of confidence measurement) would be useful to the system in order for the data to be more easily integrated into the tracker, especially if it can take on a probabilistic structure.

Programming language : The reference programming language used should be able to implement the system on the embedded system.

As well, we could divide the requirements between Functional requirements and None-Functional requirements more specified to this project.

- Functional requirements:
  - The system shall take the output of the sensors and label these data into class (Bike, Vehicle,Pedestrian).
  - The system should filter out static objects and only be concerned with moving objects.
  - The system shall classify these moving objects as belonging to one of the classes.
  - The system should be able to provide a confidence output.
- Non-Functional requirements:
  - The system shall use input data from only one Radar FMCW $24GHz$ oriented perpendicular to the road.
  - The system shall be implemented on an embedded processing target.
  - The system shall be programmed with low level of complexity.
  - The system should have an execution time that never exceeds 1 s
  - Classification accuracy more than 90

1.3 Project Objectives and Design Methodology

1.3.1 Design and Development Activities Objectives

1.3.1.1 Project Objectives

This end of studies project is about developing and evaluating a classification algorithmic solution with the purpose of identifying moving objects based on FMCW radar operating at $24Ghz$.

Objects shall be classied as belonging to one of three classes:

- Pedestrian
- Bicyclist
- vehicle

The system will be evaluated by creating different field test scenarios.

The end purpose of the classification system is to improve accuracy in the object tracking system by providing class data as an additional input.

1.3.1.2 Development Activities

The following list summarizes the project main Activities:

1. Study possible techniques to perform robust, automatic detection and classification of moving targets using FMCW Radar-based Sensor.
2. Design a smart solution for classification algorithm based on 2 techniques, the first used ML technique as for the second based on DL-techniques.
3. Measure, collect, process, and analyze a diverse corpus of target signatures.
4. Design classification algorithms based on Handcrafted features .
5. Design classification algorithms based on Range-Doppler signature.
6. Perform a detailed comparative performance analysis of support vector machine (SVM)
7. Artificial neural network (ANN) and Convolution neural network (CNN) classifiers.

1.3.2 Design and Validation Methodologies

We had adapted a 4-phase-methodolodology ;

- **Definition Phase:** In this phase we adopted a top-down approach by defining the system level requirements then the design specifications of the target algorithmic solution. In this phase, theoretical studies have be performed and current best-practices have been learned. This phase also requires the gathering of pre-existing data that can be analyzed, in order to properly define necessary functionalities and reach sufficient domain knowledge. The outputs of the definition phase constitutes what is presented in chapters 2, and chapter 3 of this report.

- **Implementation Phase:** After the definition phase, the implementation phase is performed. In this phase, code is written and implemented. We used a model-based method for developing an offline system that operates in a simulated environment. The output if this phase is the actual Development Models.

- **Testing Phase:** This third major phase consists on testing the Models in different scenarios against the respective levels of definitions and requirements. Here, a bottom-up approach is adopted: functionality is tested at the lowest useful level first, then put together and tested in groups, and finally as a complete system against the requirements defined in the first phase. The verification is done against the requirements and definitions constructed in the definition phase. This testing phase requires the gathering of more data, to evaluate the complete system in a realistic environment or appropriate experimental setup. The output of this phase constitutes chapter 4

- **Verification on validation Phase:** This project contains many sub functions, the performance of which can be evaluated separately. But since the project limited period and scope, it is not possible to spend too much time for trying to optimally assess the performance of every subsystem. In cases where there is theory or used-practice available with regards to the assessment of performance, the suggested methods can and will be used. For subsystems where such methods are not clearly available, heuristic approaches will be used instead in order to reach appropriate performance.

1.3.3 Project Planning

To achieve our goal, we established our work plan throughout the internship period as shown in the following Gantt chart which contains the chronological order of all the steps to be carried out in the project: In fact, the steps are described as the following:

- **Step 1 :**Design the global solution proposed for radar target recognition.
- **Step 2 :**Dataset preparing and analyzing
- **Step 3 :**Define the features extracted and features engineering
- **Step 4 :**Design Machine Learning-based classification solution with defined features.
- **Step 5 :**Optimize of ML Classification Models parameters.
- **Step 6 :**Code of the integrated Radar software including Matlab Phyton libraries.
- **Step 7 :**Integrate for ML classification models remaining defined features.
- **Step 8 :**Design and validation tests of ANN-based classification solution with defined features.
- **Step 9 :**Design and validation tests of Deep Learning CNN-based classification solution with automatic features extraction.
- **Step 10 :**Writing the internship report

1.4 Internship Project Work Environment

1.4.1 Hosting Company Presentation

Wevioo holding[1], established in 1998 and certified ISO 9001 V2001, specialized in IT services, consulting and technology innovation, provides its customers with digital innovation solutions that are perfectly suited to their agility, performance and development challenges. Thus, with a culture of innovation Wevioo invests in R&D to bring its clients and partners expertise in the most up-to-date technologies and the most innovative solutions.

Wevioo groups has five subsidiaries implanted in Tunis, Paris, Alger, Dubai and Abidjan. The group gathers more than 400 collaborates, 40% of whom are women. Wevioo deliver projects for customers worldwide: 60% in Europe, 25% in Africa and 15%in the Middle East.



Figure 1.2: WEVIOO footprint around the world

My internship is achieved within **EBSYS groups**[2] company the embedded systems and IoT solutions division of Wevioo holding. With strong RD culture innovation, EBSYS is particularly specialized in core hardware and software development for communicating embedded systems with a vision to provide Quality Assurance end-to-end services and expertise for innovative product design and development.

EBSYS brings 18 years' experience and an important Man-Years applying advanced digital technologies to help its customers to create, build and maintain the most innovative, reliable and cost-effective smart applications for following market segments.

- Automotive: Automatic Driver Assistance System (ADAS), Vehicle M2M/IoT Gateway, Intelligent navigation system, Car entertainment system, Vehicle thermos-electric converter, Vehicle BMS...
- Smart Grid: smart metering, street lighting control, battery management system, electricity distribution automation...
- Communication infrastructure: Radar Systems, power and spectrum efficiency of radio transmitters, wireless gateways...
- Smart City: parking lot management, intelligent video surveillance, city traffic management...
- Smart Home: multimedia door entry system, home gateway, home automation, elderly people assistance...

1.4.2 Radar Hardware Platform

Radar target classification algorithms will be implemented and tested by using the $24GHz$ multichannel radar sensor platform **Demorad** based on **Analog Devices Radar ICs**. The Demorad sensor consists, as indicated in figure below, of a single PCB with 2 transmitter/4 receiver antennas on the front side connected to the $24GHz$ RF chipset and data acquisition ADCs and DSPs on the reverse side. The Demorad uses FMCW radar to detect range and velocity of objects up to $200m$ away with a resolution of approximately $75cm$. The field of view (FOV) is approximately 120 in azimuth and 15 in elevation based on the antenna array design. By combining antennas as used in digital beam forming (DBF), the Demorad uses DBF to calculate angular information in the FOV.

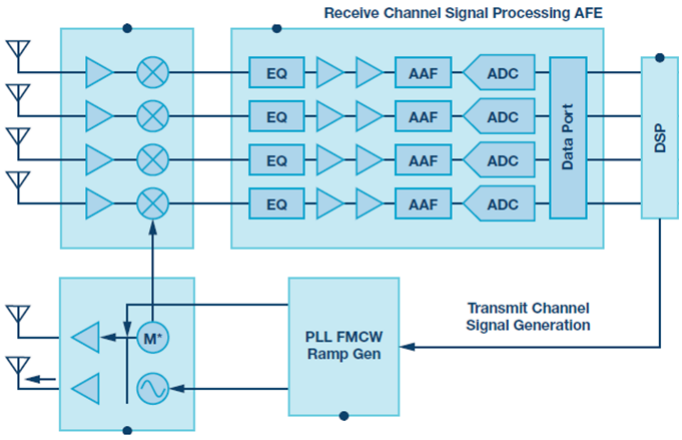


Figure 1.3: Block Diagram of the $24GHz$ DemoRad Multichannel Radar Sensor



Figure 1.4: DemoRad Kit

1.4.3 Hardware & software Tools

ML and DL training time depends on the computing power, the available memory and the central processing unit time. The work tools cover both the Hardware and the software tools including different libraries used in order to implement and validate our project algorithmic solution.

1. **Computing Hardware Ressources : :**
   - **Computer Brand :** DELL
   - **Processor :**Intel Core i7-8750H(Hexa-Core 2.2 GHz / 4.1 GHz Turbo-Cache 9 Mo)
   - **Installed Physical Memory (RAM) :** 8.00 GB
   - **GPU :** NVIDIA® GeForce® GTX 1050 Ti avec 4 Go

- **Operating System :** Microsoft Windows 10
- **Disk :** 1 TB

1. **Software Tools :**

   - **Signal Processing**: we used MATLAB, that integrates computation, visualization, and programming in an easy-to-use environment, the our project detection and the features extraction processing.
   - **Classification Processing :** We used Python that is an interpreted, high-level, general-purpose programming language. Python Created by Guido van Rossum and first released in 1991 has a design philosophy that emphasizes code readability, notably using significant whitespace. It provides constructs that enable clear programming on both small and large scales.

     We made the choice of Python for its highly ability to perform data manipulation and for the availability of various development tools which can be used to work with other systems. We used for the classification processing the following Python libraries:

     1. Pandas for data manipulation and analysis
     2. Numpy for mathematical and logical operations on data arrays
     3. Scikit-learn for building ML models
     4. Keras for building DL models
     5. Matplotlib and plotly for Data visualisation

1.5 Conclusion

In this introductory chapter, we gave an overview about target recognition for traffic monitoring application using $24GHz$ FMCW Radar. Also, the project methodology and requirements are introduced. Then, we presented the framework that we have used in this project.

CHAPTER 2

Radar Target Recognition Design Considerations

2.1 Introduction

In this chapter, reserved to the design considerations for Radar target recognition, we start by a brief survey on Artificial Intelligence (IA)-based techniques for radar target classification. The configuration of the Radar Sensor Platform, used for this project development, is exposed in the second section. In the last section,the design methodology for developing AI-based solution for Radar target classification is presented.

2.2 Survey on AI-based Techniques for Radar Target Classification

2.2.1 Radar Target Classification Processing Stages

Automatic radar target classification is needed, for the road traffic monitoring application, to distinguish between vehicles, bicycles and pedestrians. This application requires to classify the vehicles by type (car, bus, track) and by size which require an accurate and especially fast classification processing robust to the Radar data environment errors.

The processing stages of the Radar Target Recognition are represented in the block diagram of figure 2.1.

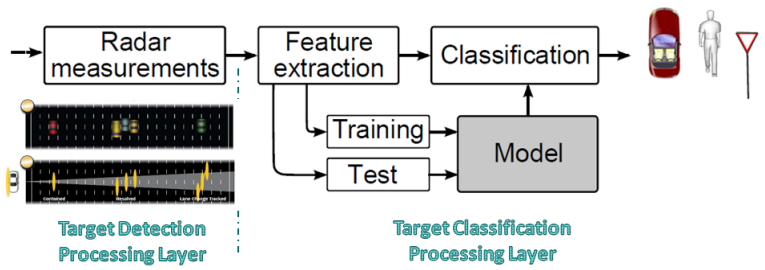


Figure 2.1: Radar Target Recognition Processing

In fact, there are 2 Artificial Intelligence approaches used for classifying the moving target. Thus, the first one is based on Machine Learning (ML) techniques where the 2nd one is based on the models of Deep Learning (DL).

The radar target classification by machine learning approach is chosen because of fast decision making required by traffic monitoring applications to ensure the surveillance of large areas and operation in multiple target environment. The main design challenge for the ML-based Radar target classification concerns the difficulty to define the best features in order to transform high-dimensional data into a low-dimensional space, with preserving as much as possible useful information about the target and removing all irrelevant data.

Figure 2.2 illustrates Radar Target Classification Processing Stages using ML techniques.



Figure 2.2: Machine Learning based Radar Target Classification Processing Stages

The second approach which is based on Deep Learning techniques is newly used in the field of Radar target classification. We tried to explore this method to show how it would work in target recognition using Range-Doppler signature. Unlike to the conventional methods that requires data processing stage, when we use Deep Neural Networks (DNN) approach, we don't need a deep knowledge of signal processing techniques to understand raw Radar data.

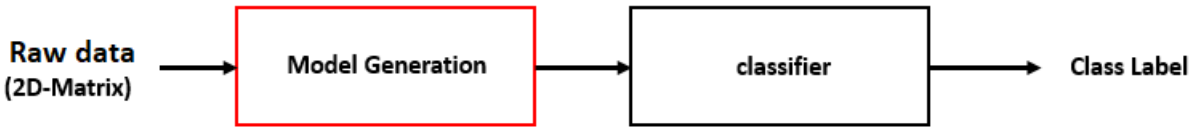


Figure 2.3: Deep Learning based Radar Target Classification Processing Stages

2.2.2 Machine Learning based Techniques

Machine learning is based on algorithms that can learn from data without relying on rules-based programming . It focuses on the development of computer programs that can access data and use it to learn for themselves. Machine learning algorithms are often categorized as supervised or unsupervised.

2.2.2.1 Supervised Learning

In Supervised learning, an AI system is presented with data which is labeled, meaning that each data tagged with the right label. After adequate training, the system can provide targets for any new input. Also, the learning algorithm can compare its output with the correct one and discover errors in order to modify the model accordingly. The supervised learning is categorized into 2 other categories which are "Classification" and "Regression".

a) Classification

Classification is technique to categorize our data into a desired and distinct number of classes where we can assign label to each class. This data set may simply be bi-class or it may be multi-class too.

Here we have the types of classification algorithms that we have used in our project.

- **Naive Bayes Classifier (NBC) :** is a classification technique based on Bayes'Theorem with an assumption of independence among predictors. In simple terms, a Naive Bayes classifier assumes that the presence of a particular feature in a class is unrelated to the presence of any other feature.

$$P(A|B) = \frac{P(B|A)\ P(A)}{P(B)}$$

- **Decision Tree Classifier (DTC) :** is a flowchart-like tree structure where an internal node represents feature(or attribute), the branch represents a decision rule, and each leaf node represents the outcome. Its training time is faster compared to the neural network algorithm. The time complexity of decision trees is a function of the number of records and number of attributes in the given data.

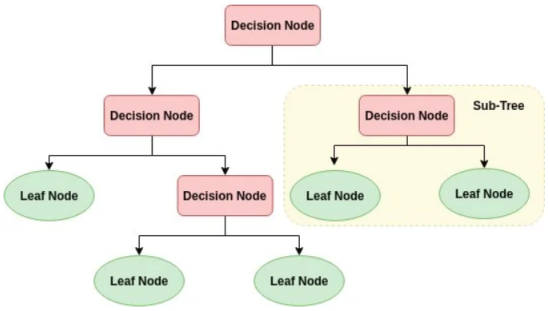


Figure 2.4: Decision Tree Classifier Concept

- **k-Nearest Neighbors (KNN) :** it assumes that similar things exist in close proximity. In other words, similar things are near to each other . In fact, a new input is classied by a majority vote of its K neighbors.

Figure 2.5 illustrates the KNN concept in the case of k = 3

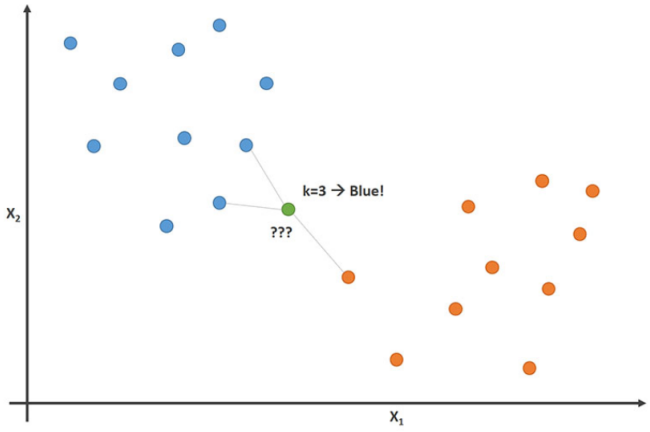


Figure 2.5: k-Nearest Neighbors Concept

- **Logistic Regression (LoR) :** is used for binary classication problems; it is a predictive analysis algorithm and based on the concept of probability . It computes the relationship between the output label and the input features, by providing discrete estimation between 1 and 0.

Figure 2.6 illustrates the steps that logistic regression goes through to give the desired output.

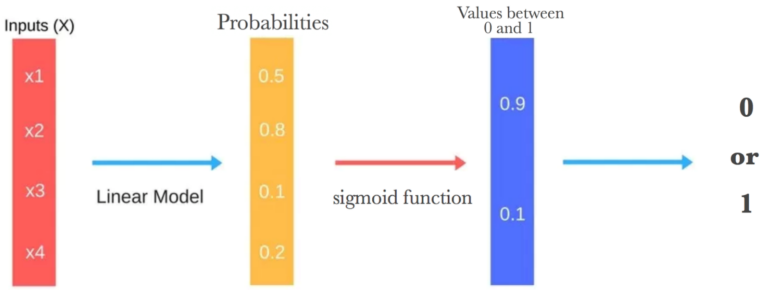


Figure 2.6: Logistic Regression Concept

- **Support Vector Machine (SVM):** is a maximum margin method that can be used for both classication and regression.One of the benets of SVM is that it is formulated as an optimization problem instead of an iterative method such as the neural network. This is benecial since it provides a mathematical insight into the structure. Another advantage is that the training can be performed using optimization techniques such as quadratic programming

As shown in figure 2.7, SVM is based on the concept of nding a hyperplane that best separates a dataset into two classes.

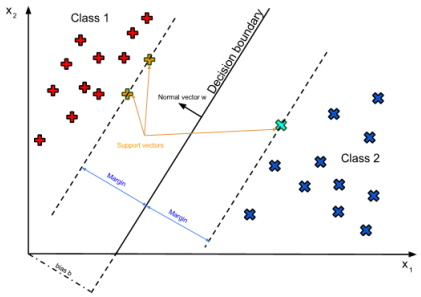


Figure 2.7: Support Vector Classifier Concept

b) Regression

The regression problem is when the target variable is continuous. This implies that the response to the issue is an amount that can be determined based on the model's inputs. The only used regression algorithm in this project is :

- **Random Forest Regression (RFR) :** is a type of additive model that combines decisions from a sequence of base models to make predictions. This broad technique of using multiple models to obtain better predictive performance is called Ensemble learning.

2.2.2.2 UnSupervised Learning

Unsupervised machine learning algorithms are used when the information used to train is neither classied nor labeled. In fact, the System does not determine the right output, but investigates the information and can draw inferences from data-sets to dene concealed constructions from unlabeled data.

The used unsupervised algorithm in this project is :

- **Principal Component Analysis (PCA):**By projecting the data into a lower dimensional space consisting of the first two or three principal components, a better overview is obtained. Thus, the PCA method will be used in determining whether the nal classifier performs as can be expected. If there is a clear correlation between the separability of particular subsets of data (for example, a low separability of the pedestrian and the bicycle class), then it can be expected that the classication system will have trouble distinguishing between these particular classes.

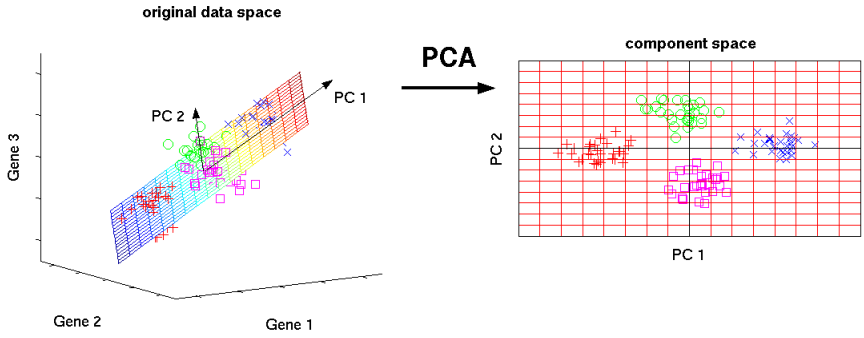


Figure 2.8: Principal Component Analysis Concept

2.2.3 Deep Learning based Techniques

Deep learning is a type of machine learning built on a deep hierarchy of inter connected "neural network" layers; it takes massive amounts of data and determines the common rules and features associated with the data.

the used algorithms in this approach are:

- **Feedforward Neural Networks :** also called Artificial Neural Networks (ANN), is multi-layer fully connected neural net that look like the figure 2.9. They consist of an input layer, multiple hidden layers, and an output layer. Every node in one layer is connected to every other node in the next layer. The network is deeper by increasing the number of hidden layers and number of hidden neurons in each layer.

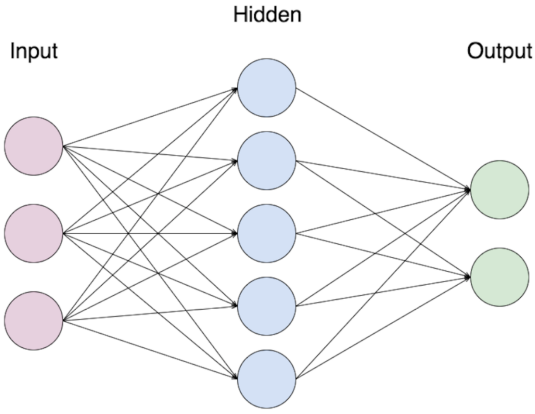


Figure 2.9: ANN Architecture with one hidden layer

- **2D-Convolutional Neural Networks (2D-CNN):**a CNN operates well within the data to identify simple patterns that will then be used to create more complicated patterns within higher layers. The main goal of using such algorithm that it does not require a

feature engineering step as they learn high-level features from data in an incremental manner.

A simple 2-D Convolutional Neural Network (CNN) model is designed using keras with tensorflow libraries backend Using Range-Doppler spectrum as an input. We choose 2D convolution because the movement of the filter across the spectrum happens in two dimensions.

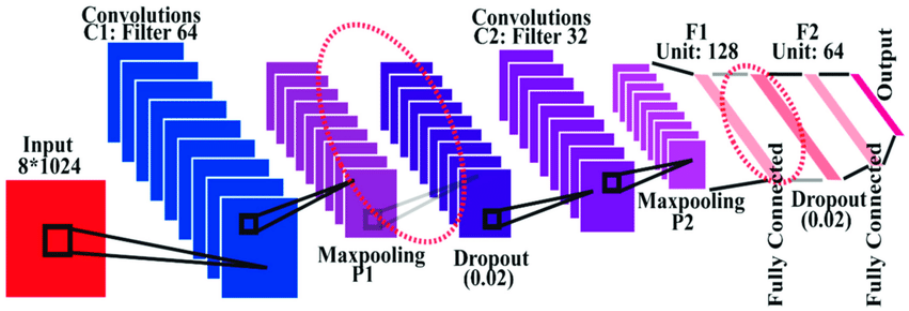


Figure 2.10: Architecture of CNN

2.3 Radar Sensor Platform Configuration

2.3.1 DemoRad 24 GHz Radar Transceiver Configuration

The DemoRad Radar platform is built, as illustrated in figure **??**, by using the following chipsets from Analog Devices:

- BF706: Digital Signal Processor for the implementation of the embedded software to control the RF frontend and to process the measured radar signals of the receive channels.
- ADF4159: Direct Modulation/Fast Waveform Generating 13 GHz Frequency Synthesizer to generate the FMCW transmit signal.
- ADF5901: 2-Channel transmitter: 24 GHz VCO and PGA with 2-Channel PA Output
- ADF5904: 4-Channel 24 GHz Receiver Downconverter.
- ADAR7251: 4-Channel 16-Bit Continuous Time Data Acquisition ADC. It is used to amplify and sample the measured IF signals of the receiver

The Radar signals are processed in the DSP and the results can be accessed by a Host processor with a USB 2.0 or a CAN interface.

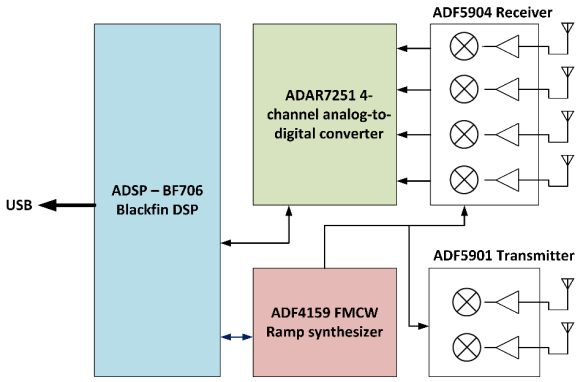


Figure 2.11: DemoRad Radar Transceiver Components

Radar Transceiver Component	Configuration Parameters	Programmed Configuration
ADF4159 Direct Modulation/Fast Waveform Generating	Waveform type RF bandwidth	Continuous sawtooth ramp 13 GHz Chirp parameters (see table II.2-1)
ADF5901 2-Channel RF Transmitter	RF Output Power VCO Output signal bandwidth	8 dBm 24 GHz to 24.25 GHz 250 MHz
ADF5904 4-Channel RF Receiver	Rx channel gain RF signal bandwidth Noise figure	22 dB 250 MHz 10 dB
ADAR7251 4-Channel, 16-Bit, Continuous Time Data Acquisition ADC	Input signal bandwidth Sampling rates LNA & PGA gain Equalizer (cutoff) Internal oscillator input	500 kHz 1, 1.2 and 1.8 MSPS 45 dB 32 kHz, 37 kHz, 45 kHz, or 54 kHz 16 MHz to 54 MHz

Table 2.1: DemoRad Radar hardware elements

Table 2.2 gives the values of the chirp parameters.

Parameter	Description	Value
f_{Start}	Start frequency	23.9 GHz
f_{Stop}	Stop frequency	24.3 GHz
T_{Ramp}	Upchirp duration	256 µs
f_s	Sampling frequency	1 MHz
N	Number of samples for one chirp	256
N_p	Number of chirps	128

Table 2.2: Radar Chirp parameters

- **DemoRad Transceiver Configuration Software**

The DemoRad Transceiver configuration software via Python or MATLAB codes is composed, as illustrated in figure 2.12, of the following modules:

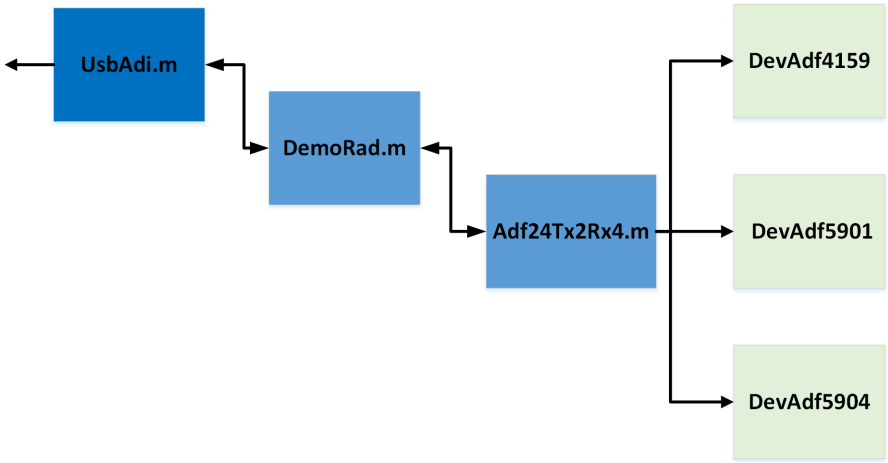


Figure 2.12: DemoRad Transceiver Configuration Software

- **UsbAdi.m**

  Class UsbAdi is the handle class that serves information directly to the defined USB interface. This class defines a set of key functions in terms of communication with the DemoRad kit board: CmdBuild, CmdExec, CmdReadData, CmdRecv, CmdSend, DataRead and Data Send. These functions give the specific format to the information that is transferred to the board and also give orders to the board in order to read content from its memory. The bus width of the DSP module is 32 bit and thus all read/write

orders have to be translated into a 32 bit-long word containing both operation codes and memory addresses.

- **DemoRad.m.m**

  The Demorad.m class is the initialization of the DemoRad board and its parent superclass is UsbAdi. This class defines a long list of methods but only those strongly related with the project are detailed here under:

  - BrdGetChirp is a key function in the radar setup. This function commands the module to send, sample and return the data from a set of chirp signals. The length of the chirp frame is defined by parameters StrtPos and StopPos.
  - BrdGetCalDat and BrdGetCalInf are two methods that return the calibration data stored in the EEPROM memory of the DSP. This data is used to allow MIMO angular estimation, and can be re-written if a new calibration is performed.
  - DspSetMimo enables the MIMO mode of the module. This mode consists on alternating $TX1$and $TX2$ for each chirp.
  - BrdRst resets the board to its default status. It is required to reset the internal registers after each measurement in order to load different configuration parameters.

- **Adf24Tx2Rx4.m:**

  Class $Adf24Tx2Rx4$ is the child class of DemoRad and apart from all the inherited content, it defines three public sub-objects named Adf_Rx,Adf_Tx and Adf_Pll. As private properties, this class specifies default configuration parameters that are needed to initialize the board when it is powered up, although they are reconfigured before each measurement routine. Some of this parameter for example are Rf $fStrt = 24e9GHz$ or Rf $fStop = 24:25e9Ghz$.

2.3.2 DemoRad 24 GHz Radar Target Detection Architecture

This part investigates multiple signal processing routines for target detection that will be implemented on the host processor. While moving target indication filtering and angle computing are well-known concepts from literature, Range-Doppler effect is a newly proposed method for enhanced parameters estimation of moving targets. The purpose of this stage is to give an accurate estimation parameters to differentiate multi-target in different scenarios. Figure 2.13 illustrates the architecture of detection processing stages.

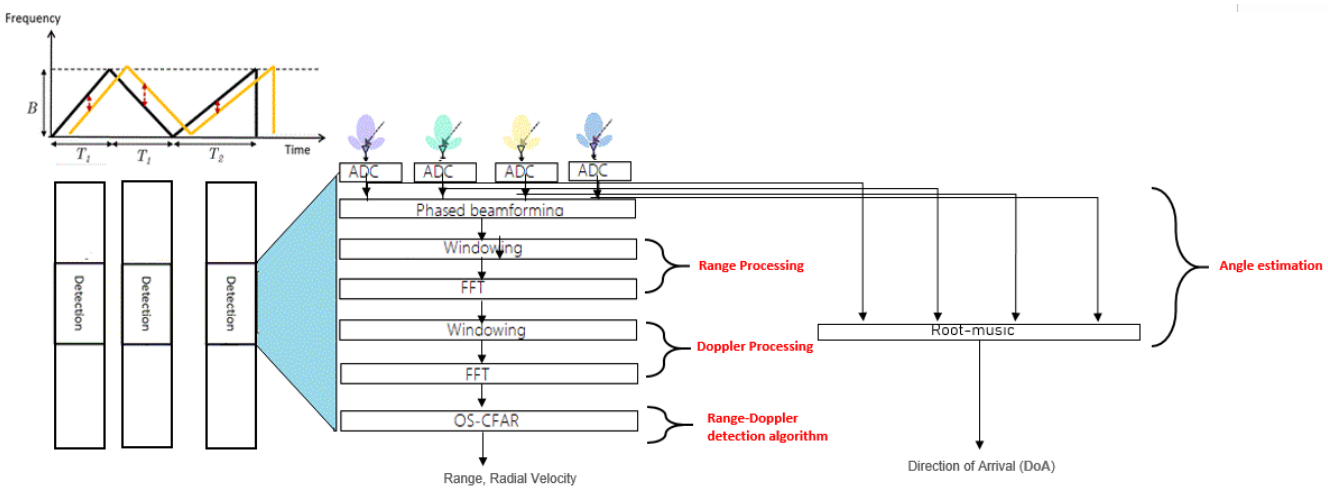


Figure 2.13: Target Detection Processing Stages Architecture

- **Beam forming processing :** Digital PhaseShift beamforming (DBF) algorithm is applied to combine received power signal arriving from 4 antennas in time domain.
- **Range processing :** : For the resulted signal from the previous stage, 1-D windowing and 1-D fast Fourier transform (FFT) are applied. Range processing is interleaved with the active chirp time of the frame.
- **Doppler processing :** After Range Processing stage , 2-D windowing and 2-D FFT are applied .For each detected point in range and Doppler space, reconstruct the 2-D FFT output(RD map) with Doppler compensation.
- **Range-Doppler detection algorithm :** Ordered Statistic CFAR (OS-CFAR) detection algorithm run on the range-Doppler power mapping to find detection points in range and Doppler space.
- **Angle estimation :** A Root-MUSIC algorithm returns one angle from the 4 incoming signals.

2.3.3 DemoRad 24 GHz Radar Target Detection Data Format

The explanation of the Radar Sensor Platform Configuration in the previous section is necessary in order to fully understand the measurement routines presented at this point.

Once the board is ready to measure, the BrdGetData function can be called and the data of a measurement will be saved into the destination variable. This function will output a 256×4

matrix when the length of the frame is one and a $128 \times 256 \times 4$matrix if the frame length is 128. Here each row is a time instant (i.e the first 256 rows are the 256 samples of one chirp) and each column is one of the four different receiver channels.

In figure 2.14 illustrates the output of a single chirp measurement .

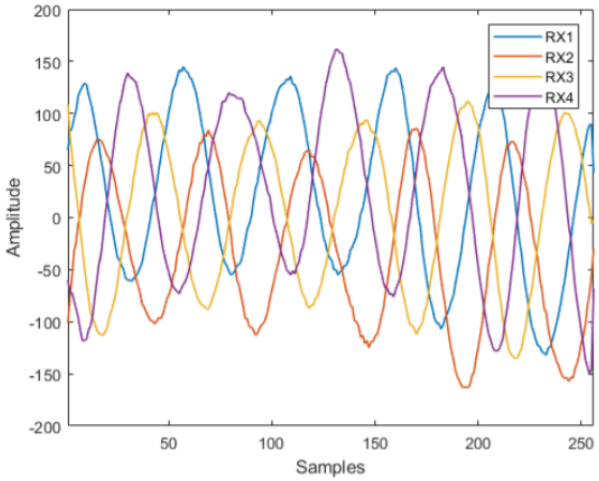


Figure 2.14: Time-domain representation of a single chirp measurement

In this project, the standard data format has been defined as a $256 \times NumChirps$ matrix for each of the 4 channels.To recorder a sequence of frames, we used a measure loop.However, the matrix where the output data is loaded after calling BrdGetData is overwritten at each iteration. This can be solved by means of concatenating the output data after each iteration.

2.4 Design Methodology for the Radar Target Classification Solution

2.4.1 AI-based Radar Target Classification Architecture

To construct the learning model for this project application, the steps detailed below should be followed. Figure 2.15 illustrates the general architecture of the proposed Radar Target Classification solution.

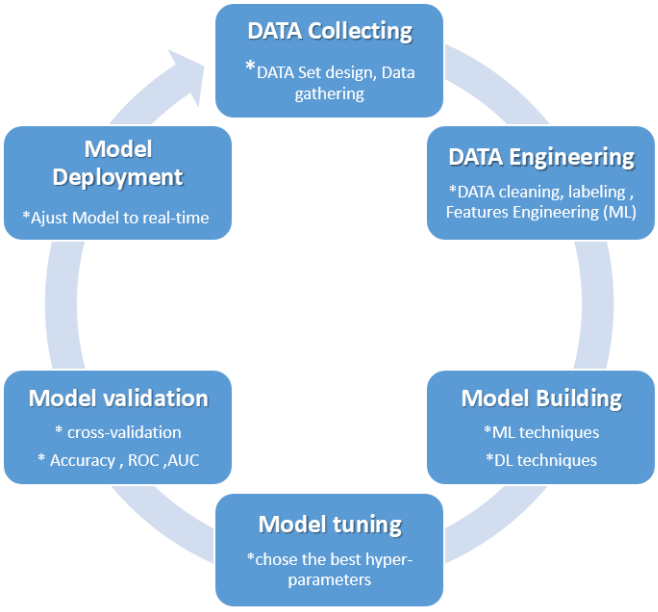


Figure 2.15: Radar Target Classification Project Work-flow

To apply Radar Target recognition processing to the Data generated by the DemoRad kit we implemented the processing stages with 2 approachs ML & DL represented in the figures 2.16 and 2.17 below.

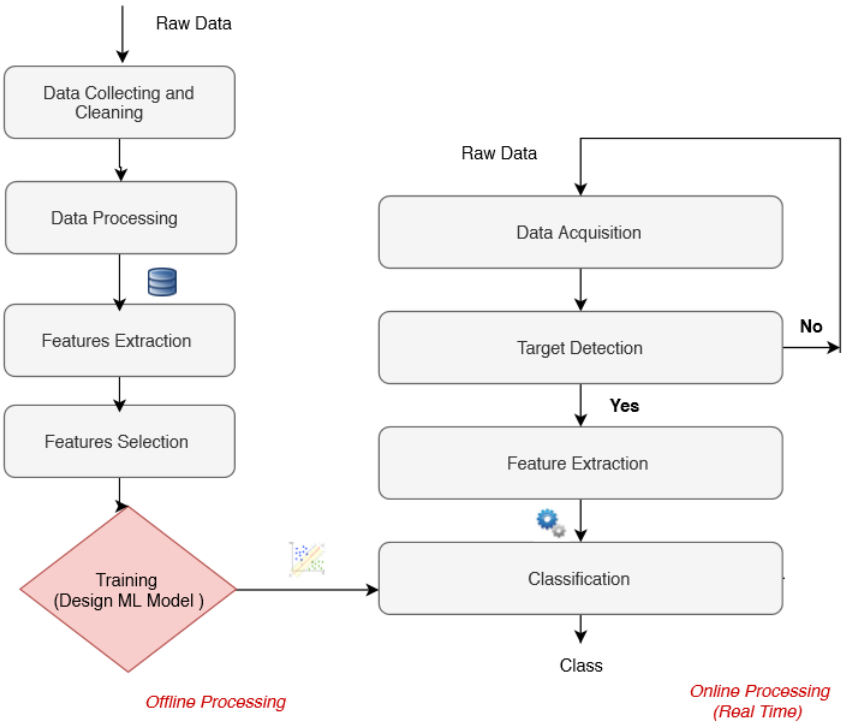


Figure 2.16: Radar Target Recognition Processing using ML

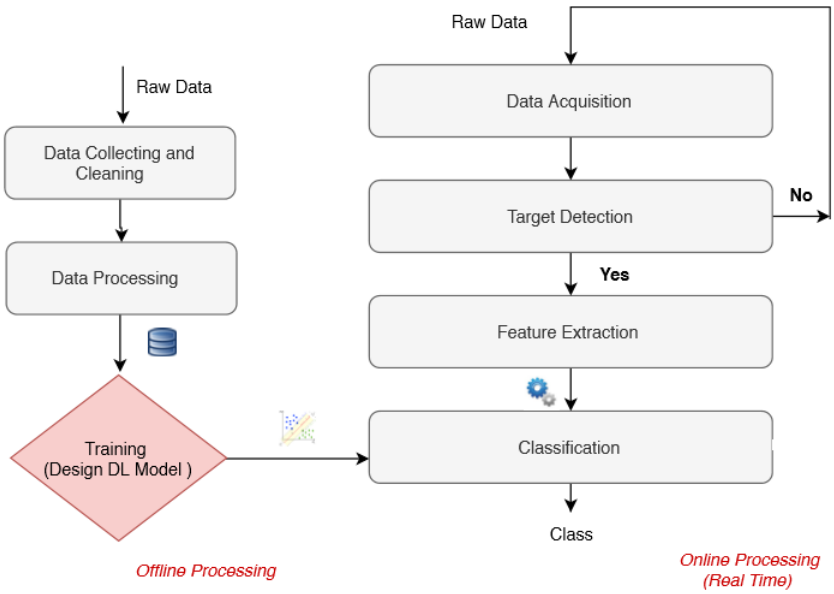


Figure 2.17: Radar Target Recognition Processing using DL

2.4.2 Data Set Building

For exploring the two ML DL techniques, two databases were prepared, one for the ML method and the other for the DL method.

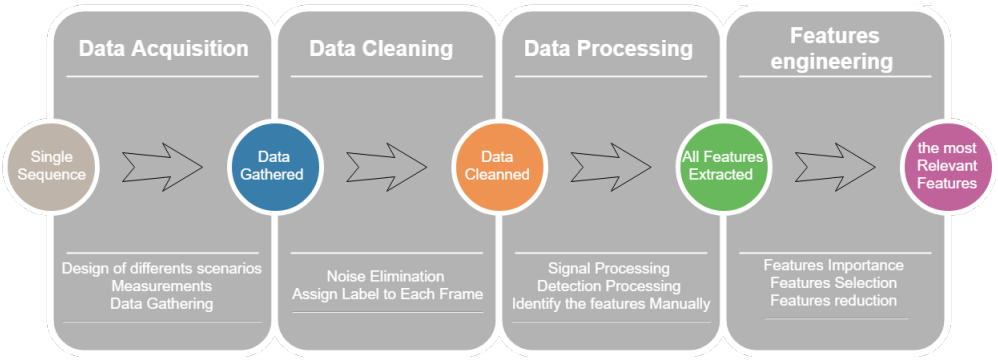


Figure 2.18: Training Data Base ML Construction

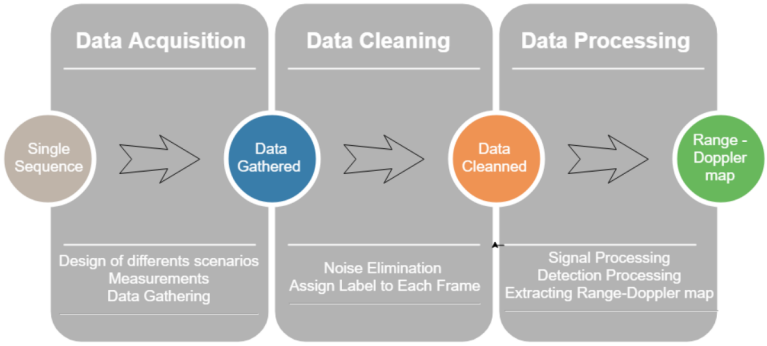


Figure 2.19: Training Data Base DL Construction

In order to implement the Radar Target Classification architecture we start by constructing the training data base

- **Data acquisition**

  In order to build a robust classification system, it is necessary to construct a rich dataset made up of multitude of classes. At the first step, we started by designing different test scenarios for each class. These scenarios are divided between those in a controlled environments, and others in an uncontrolled environment. In the second step, we gathered the data in different environments. Video recording was made during the whole session. The recorded video sequences allow us later to label the different classes of moving target.

- **Data Cleaning**

  TThe second stage of data engineering is data cleaning. This processing consists in seeing the noise distribution (no target present) compared to the others frame where there is a target. So, we plotted signal amplitude and detected the amplitude that corresponds to target based on Range-Doppler map and finally we assigned to each frame a label to define the class based audio-visual documentation recorded during data gathering process.

- **Data Processing**

  In this stage, the raw radar signals enter the processing unit, consecutively passing through all necessary signal processing steps. Different levels of data abstraction and representation can be identified, which range from low level time signals up to complex environmental models. For the machine learning approach, we worked on the extraction

of the specific features for each target. For deep learning processing we stopped at the 2D-FFT stage. Indeed, the added value of deep learning is to automatically extract the features by exploiting the raw signal without resorting to different signal processing techniques.

- **Features Engineering**

  Features Engineering implies exploring the usage of different features to describe objects, as well as using statistical methods to determine characteristics of the labeled data. It can be performed after each level of data processing . In this process, different feasible features that can be extracted from the training data are examined. The best ones are selected to be part of the feature extraction. This is described more in detail in section 3.4, Practical selection and analysis of object descriptions

For each frame, several features are computed from the radar signal to characterize the moving target described in the following section. We recall that features resumes the proprieties of the signal into a set of descriptors. For the sake of simplicity, let Fn denote either signal after the different blocks of pre-processing stage. For each Fn, the following time-domain features have been considered in each frame: maximum, mean, variance (VAR), standard deviation (STD). The frequency domain features used are extracted after OS-CFAR stage and 2D-FFT blocks. After this step, feature selection take place to identify the most important features by ranking them according to a computed relevance score. For decrease the correlation between features, we applied the PCA algorithm. Figure 2.20 shows the feature extraction concept.

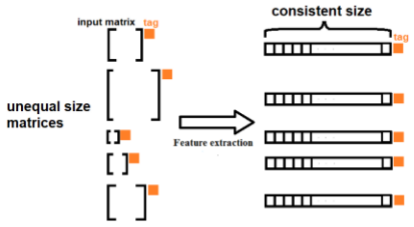


Figure 2.20: Feature extraction Concept

The feature selection concept is shown in the gure 2.20.



Figure 2.21: Feature Selection Concept

2.4.3 LearningClassification Model Building and Evaluation

- **Classification Model Building**

The classification procedure begins with the acquisition of a frame of raw data. Basic pre-processing steps such as high pass filtering are performed in this stage. The data is then fed into a target detection algorithm. If the detection algorithm decides that a target is present, the target features are extracted from the data. In the feature extraction stage, the data frame is processed to quantify the various target features used for classification. In the training phase, feature selection would follow feature extraction. Feature selection seeks to remove any highly correlated features (features containing redundant information) from the extracted feature set. After feature extraction, the classification stage utilizes statistical models and thresholds created during the training procedure for discrimination.

The classification step produces the estimated target class, and then the cycle repeats for subsequent data frames.

- **Evaluation Metrics**

First, there are 4 important terms to be considedred:

True Positives : The cases in which we predicted YES and the actual output was also YES.

True Negatives : The cases in which we predicted NO and the actual output was NO.

False Positives : The cases in which we predicted YES and the actual output was NO.

False Negatives : The cases in which we predicted NO and the actual output was YES.

$$Accuracy = \frac{TruePositives + TrueNegatives}{Totalnumberofpredictionsmade}$$

- **Accuracy :**Classication accuracy is the number of correct predictions made as a ratio of all predictions made. This is the most common evaluation metric for classication problems
- **Confusion Matrix:** this matrix describes the complete performance of the model. Each row of the matrix reflects the cases in a class predicted, whereas each column represents the cases in a class (or vice versa). The name derives from the fact that it is simple to see whether the scheme confuses two classes.
- **Area Under ROC Curve :**Area under ROC Curve (or AUC for short) is a performance metric for binary classification problems [Jason Brownlee, 2019]. The AUC represents a model's ability to discriminate between positive and negative classes. An area of 1.0 represents a model that made all predictions perfectly. An area of 0.5 represents a model as good as random. ROC can be broken down into sensitivity and specificity. A binary classification problem is really a trade-o between sensitivity and specificity.

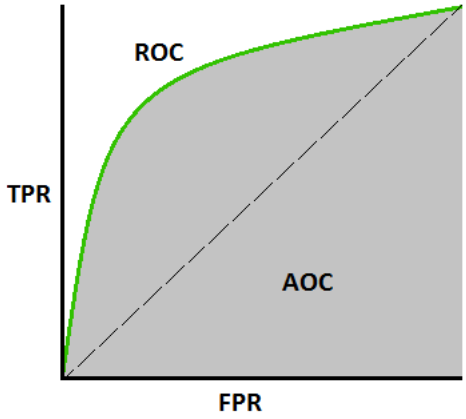


Figure 2.22: ROC Curve With High AUC

Though, an extension of the ROC curve to multiclass forecast problems is explored in [WANDISHIN and MULLEN, 2008] where we could treat the multiclass as a binary classification problem and to consider each class vs all the other classes and calculate the operating points for each class and then average it out for the entire selected classifier.

- **Classification report:** it shows a representation of the main classification metrics on a per-class basis. This gives a deeper intuition of the classier behavior over global accuracy which can mask functional weaknesses in one class of a multi-class problem.Using this terminology the metrics are defined as follows:

Precision : Accuracy of positive predictions.

$$Precision = \frac{TruePositives}{TruePositives + FalsePositives}$$

Recall(sensitivity or true positive rate) : Fraction of positives That were correctly identified.

$$Precision = \frac{TruePositives}{TruePositives + FalseNegatives}$$

F1 Score (F-Score or F-Measure) : A helpful metric for comparing n classifiers. F1 Score takes into account precision and the recall. It is created by finding the the harmonic mean of precision and recall that the best score is 1.0 and the worst is 0.0.

$$F1 = \frac{2 * (Precision * Recall)}{Precision + Recall}$$

Support : Support is the number of actual occurrences of the class in the specied data-set.mbalanced support in the training data may indicate structural weaknesses in the reported scores of the classifier and could indicate the need for stratified sampling or rebalancing.

2.5 Conclusion

Throughout this chapter,we detailed the design of the solution by presenting the different stages of the project, some ML and DL concepts and models that will be used to resolve the problematic. In the next chapter, we will present the data set specifications, and we will expose the chain of data engineering that lead to extract the most significant features.[**?**]

CHAPTER 3

Field Test Campaign and Data Set Analysis

3.1 Introduction

This chapter is dedicated to describing the whole processing to get the data set that we fed into the classifier. We remind that this project based on creation of two dataset, the first one for ML and the second one for applying DL techniques. We start by presenting Dataset design, then, we will illustrate data processing for the two databases. Finally, we will provide different steps of feature engineering to get the most relevant dataset for ML approach.

3.2 Field Test Scenarios Definition

3.2.1 Data Collection Environment

In order to perform supervised learning and to gain data domain knowledge, the first process needed is the dataset building. For this purpose, we start by establishing the environment of the dataset using the **DemoRad** platform.

Figure 3.1 shows the mobile data collection platform built to facilitate the data collection process.

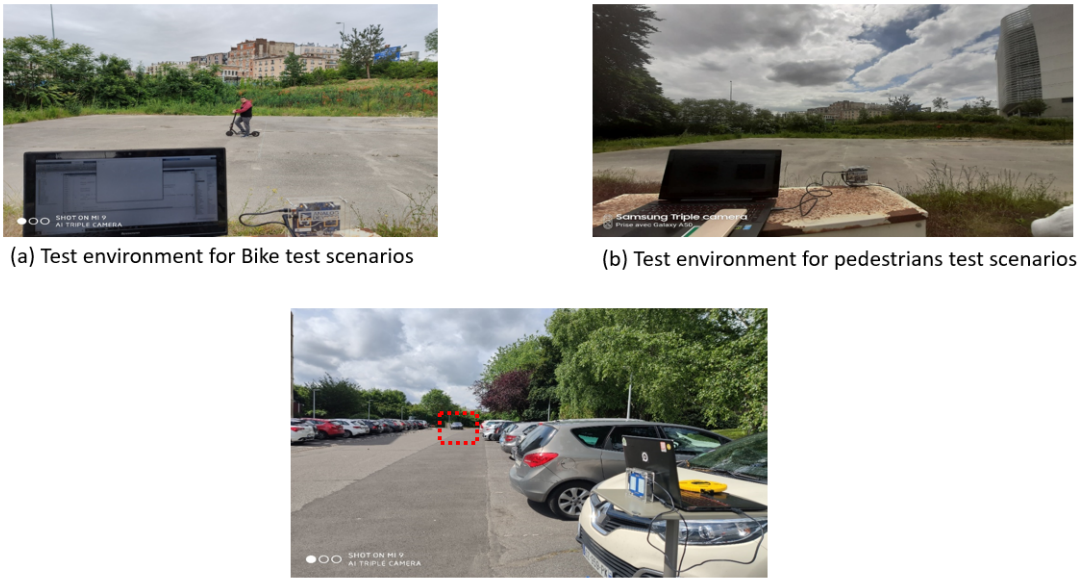


Figure 3.1: Mobile data collection platform

As seen in figure 3.1 above the radar hardware is encased in an enclosure directly in front of the target. The mobile phone camera is on the right of the radar for recording videos to label different classes of moving target (Vehicle, Pedestrians, Bike). The computer behind the radar hardware stores the large amount of data acquired from simultaneous acquisition of radar signatures. By having this mobile data collection platform it is easy to move it between different field test locations.

For enriching the database, we processed by gathering data from controlled environment where we have designed different scenarios and others in an uncontrolled environment where we received data from all target in radar field of view. Most of the radar tests were performed in Evry, Paris, France.

Figures 3.2 illustrates the measurement in controlled environment.



(a) Test environment for Bike test scenarios

(b) Test environment for pedestrians test scenarios

(c) Test environment for Vehicle test scenarios

Figure 3.2: Measurement Set-up for Controlled Environments

Figure 3.3 shows some examples of Radar sensor positions for uncontrolled environment tests.



Figure 3.3: Measurement Set-up for Uncontrolled Environments

3.2.2 Data Collection

3.2.2.1 Controlled Environments

In order to acquire labeled data from each of the three classes Vehicle, Pedestrians and Bike, several test scenarios using the DemoRad platform have been realized. Tables 3.1, 3.2 and 3.3 indicate test scenarios description for the three types of target classes.

Number of Test Scenarios	Test Scenarios Description
14	- Pedestrian moves straight towards the radar at a distance of 5m , walking
	- Pedestrian moves straight towards the radar at a distance of 5m , running
	- Pedestrian moves perpendicular to the radar at a distance of 5m, walking
	- Pedestrian moves perpendicular to the radar at a distance of 5m, running
	- Pedestrian moves diagonally (from left to right) to the radar at a distance of 5m , walking
	- Pedestrian moves diagonally (from left to right)to the radar at a distance of 5m , running
	- Pedestrian moves diagonally (from right to left) to the radar at a distance of 5m , walking
	- Pedestrian moves diagonally (from right to left) to the radar at a distance of 5m , running
	- Pedestrain moves around the radar in a circle of approximately five meters diagonal , walking slowely
	- Pedestrain moves in a circle of approximately five meters diagonal , jogging
	- Pedestrain moves in a circle of approximately five meters diagonal , running
	- Pedestrain moves in a chaotic manner
	- Pedestrain moves around the radar in a circle of approximately 10 m diagonal , jogging
	- Pedestrain moves around the radar in a circle of approximately 10 m diagonal , running

Table 3.1: Test Scenarios for Pedestrians Target Class

Number of Test Scenarios	Test Scenarios Description
10	- Bicyclist moves straight towards the radra a distance of 10 m, normal speed
	- Bicyclist moves straight towards the radra a distance of 10 m, maximal speed
	- Bicyclist moves perpendicular to the radar at a distance of 10m, normal speed
	- Bicyclist moves perpendicular to the radar at a distance of 10m, maximal speed
	- Radar placed in a 45 degree angle, trigonometric direction , Bicyclist moves in normal speed
	- Radar placed in a 45 degree angle, trigonometric direction , Bicyclist moves in maximal speed
	- Bike moves around the radar in a circle of approximately 20 m diagonal , normal speed
	- Bike moves around the radar in a circle of approximately 20 m diagonal , maximal speed
	- Bike moves diagonally in a different speed a
	- Bike moves in a chaotic manner

Table 3.2: Test Scenarios for Bike Target Class

Number of Test Scenarios	Test Scenarios Description
4	- Vehicle runs at 20 km/h , perpendicular to the radar
	- Vehicle runs at 25 km/h , perpendicular to the radar .
	- Vehicle runs at 30 km/h , perpendicular to the radar .
	- Vehicle runs at 35 km/h , perpendicular to the radar .

Table 3.3: Test Scenarios for Vehicle Target Class

To summarize the results of this measurement campaign, Table 3.4 illustrates the details of the data collection results for the three classes.

Test ID	Test environment	Test targets	Test duration (min)	Generated Measures (frames)
T-01	Location: basketball Court Range: 15m 14 scenarios (described above) Time: Morning	Pedestrians	70.5	2350
T-02	Location : basketball Court Range: 15m 10 scenarios (described above) Time: evening	Bike	69	2300
T-03	Location : Parking of telecom Sud PARIS Range: 10m 4 scenarios (described above) Time: Morning	vehicle	24	800

Table 3.4: Measurement Information for controlled environment

3.2.2.2 Uncontrolled Environments

For the second measurement campaign (In a UnControlled Environment) we choose a variety of places to collect the data but most of the data collected for training dataset, are gathered in the city of Evry, Paris in different areas (Crossroads, highways, near the University Campus...).

Table 3.5 describes the test scenarios captured in uncontrolled environment:

Test ID	Test environment	Test targets	Test duration (min)	Generated measures
T-01	Location: Outdoor public road – 1 way Range: 6m Different speeds: 30, 40, 50, 60 km/h Time: Night time	Cars	24	800
T-02	Location: Outdoor public road – 1 way Range : 5 m Time: Morning	Cars Pedestrians Bus	30	1000
T-03	Location: Parking Range : 6 m Time: Night	Cars	21	700
T-04	Location: Outdoor public road – 2 ways Range : 10 m Time: Morning	Cars Pedestrians Bus	57	1900
T-05	Location: Outdoor public road – 1 way Range : 8 m Time: Morning	Pedestrians	21	700
T-06	Location: Outdoor public road – 1 way Range : 4 m Time: Night	Bike	36	1200
T-07	Location: Outdoor public road – 1 way Range : 4 m Time: Morning	Moto-bike	6	200
T-08	Location: Outdoor public road – 2 ways Range : 6 m Time: Morning	Cars Pedestrians Bike	30	1000

Table 3.5: Measurement Information for uncontrolled environment

These data are collected using two Radar transceiver control software. In the first configuration the MIMO (2Tx/4RX) mode is activated with the "AN24.07.m" Matlab module. In the second configuration one single transmit channel and 4 receive channels ($1Tx/4RX$) mode is activated with the"AN24.06.m" Matlab module. For both modes, we used the Matlab function "Brd.GetData()" to gather the row data.

3.3 Data Set Building

3.3.1 Data Set Cleaning

After gathering data, we started by cleaning data collected, this processing consists in seeing the noise distribution (where there is no target present) compared to the others frame where there is a target. So, we plotted signal amplitude and detected the amplitude that corresponds to target based on Range-Doppler map.

- **Pedestrian Class** For the data corresponding to Pedestrians Class, we have 14 scenarios in the controlled environment, each scenario is a sequence of frame of different lengths. The figure 3.4 shows length of sequence for each scenario

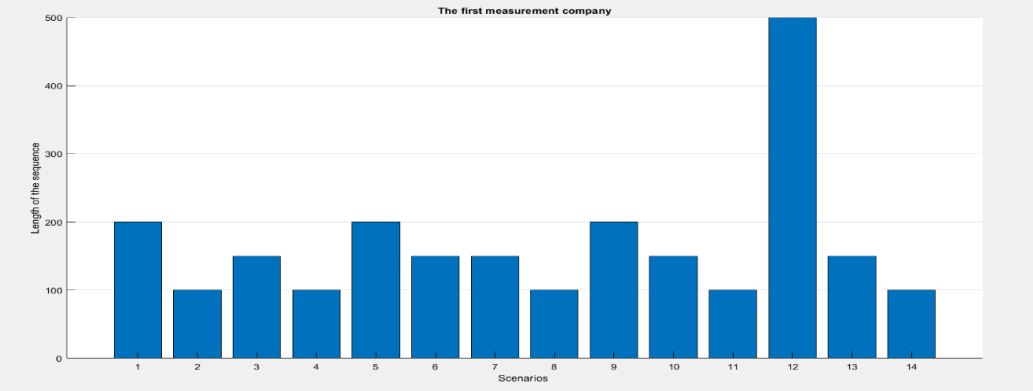
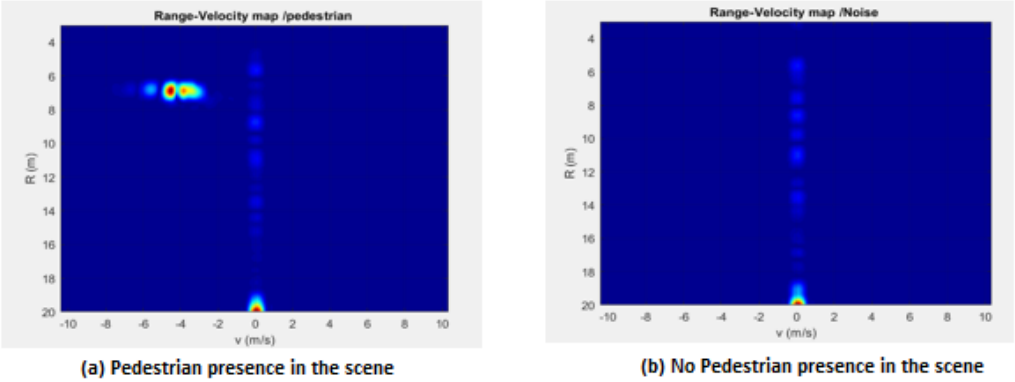


Figure 3.4: Sequence length for Pedestrians Class

Range-Doppler is used for identifying the presence of the moving target. Figure 3.5 illustrates the difference between the presence of pedestrian and silent time where there is no target in the field of view of the radar.



(a) Pedestrian presence in the scene

(b) No Pedestrian presence in the scene

Figure 3.5: Range Doppler Map for Pedestrian Class Measured Dataset

Table 3.6 shows the repartition of useful sequences containing pedestrian Radar signature.

Measurement files	Lengths of the sequence (frame)	Useful sequence		
		Nb_useful	ID_Start	ID_Stop
« Sceanrio_1.mat »	200	108	92	200
« Sceanrio_2.mat »	100	85	15	100
« Sceanrio_3.mat »	150	147	3	150
« Sceanrio_4.mat »	100	95	5	100
« Sceanrio_5.mat »	200	172	4	175
« Sceanrio_6.mat »	150	141	7	147
« Sceanrio_7.mat »	150	134	16	150
« Sceanrio_8.mat »	100	72	28	100
« Sceanrio_9.mat »	200	200	1	200
« Sceanrio_10.mat »	150	150	1	150
« Sceanrio_11.mat »	100	100	1	100
« Sceanrio_12.mat »	500	500	1	500
« Sceanrio_13.mat »	150	149	1	149
« Sceanrio_14.mat »	100	100	1	100
14 fichiers	2350	2159		

Table 3.6: Repartition of Useful Sequences for Pedestrians Class

Figure 3.6 illustrates the measured reflected power of each frame for the Pedestrians Class test scenario. The red signal corresponds to all the frames concatenated before the cleaning processing and the one in blue corresponds to useful frames after the processing.

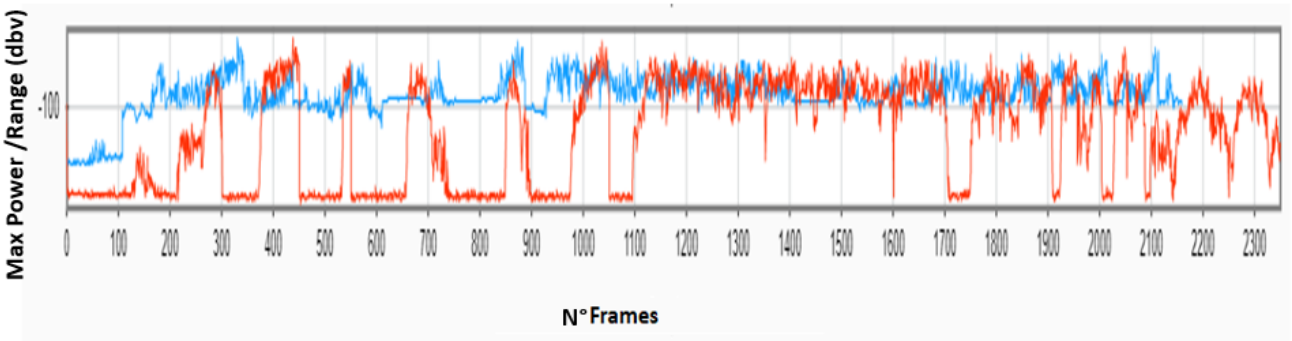


Figure 3.6: Reflected Power Repartition of Radar Frames before and after Cleaning Processing for Pedestrians Class

- **Bike Class**

For the data corresponding to Bike Class, we have 10 scenarios in the controlled environment, each scenario is a sequence of frame of different lengths. The figure 3.7 shows length of sequence for each scenario.

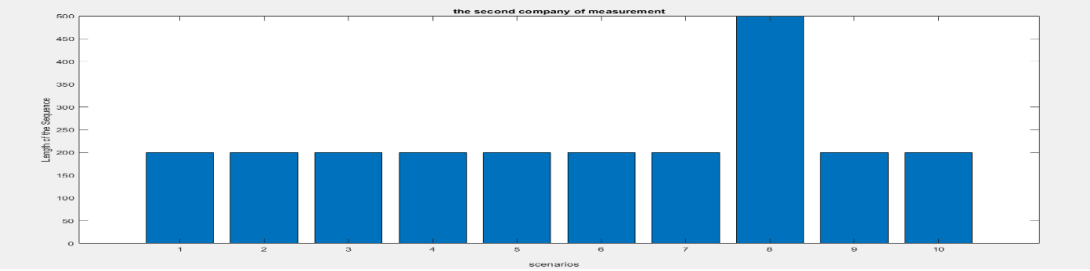
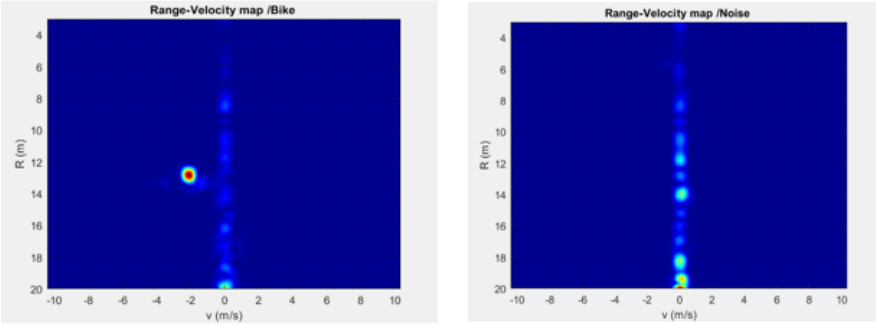


Figure 3.7: Sequence length for Bike Class

Range-Doppler is used for separating between the useful frame where the target is present and the other frames corresponding to the noise.



(a) Bike present in the scène **(b) No Bike present**

Figure 3.8: Range Doppler Map for Bike Class Measured Dataset

Table 3.7 shows the repartition of useful sequences containing bike Radar signature.

Measurement files	Lengths of the sequence (frame)	Useful sequence		
		Nb_useful	ID_Start	ID_Stop
« Sceanrio_1.mat »	200	32	13	28
			85	100
« Sceanrio_2.mat »	200	30	1	30
« Sceanrio_3.mat »	200	50	1	50
« Sceanrio_4.mat »	200	45	1	45
« Sceanrio_5.mat »	200	33	1	33
« Sceanrio_6.mat »	200	44	4	47
« Sceanrio_7.mat »	200	176	1	35
			60	200
« Sceanrio_8.mat »	500	428	1	65
			90	300
			320	360
			370	480
« Sceanrio_9.mat »	200	18	27	44
« Sceanrio_10.mat »	200	91	1	50
			60	100
10 fichiers	2300	947		

Table 3.7: Repartition of Useful Sequences for Bike Class

Figure 3.9 illustrates the result for the Bike Class. The red signal corresponds to all the frames concatenated before the cleaning processing and the one in blue corresponds to useful frames after the processing.

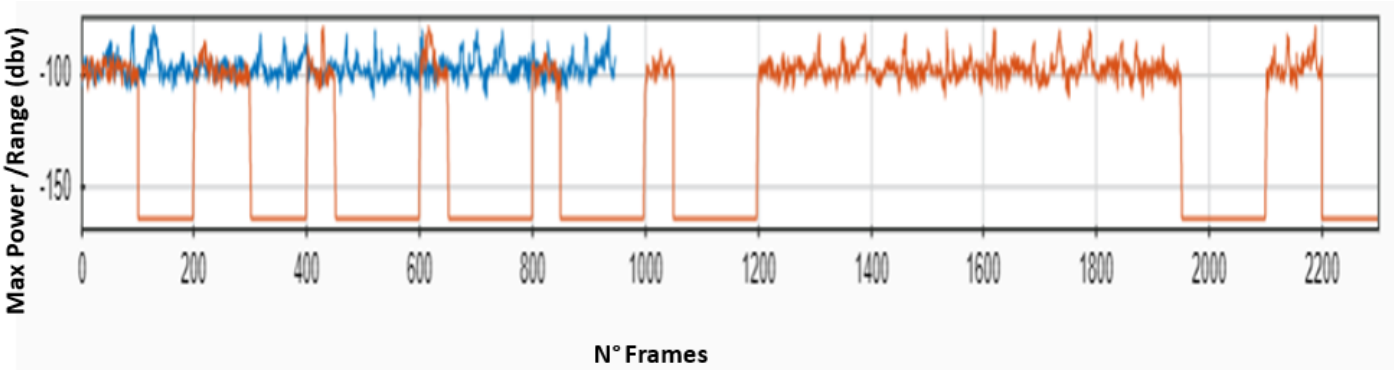


Figure 3.9: Reflected Power Repartition of Radar Frames before and after Cleaning Processing for Bike Class

- **Vehicle Class**

  For the data corresponding to Vehicle Class, we have 4 scenarios in the controlled environment, each scenario is a sequence of frame of different lengths. The figure 3.10 shows

length of sequence for each scenario.

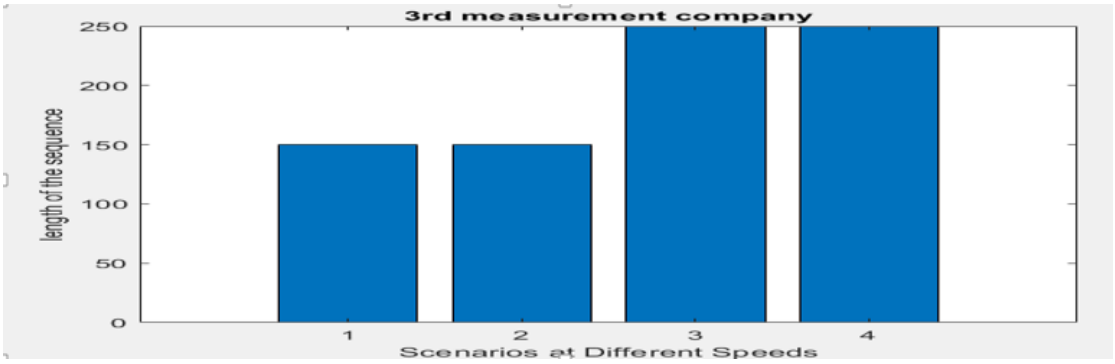


Figure 3.10: Sequence length for Vehicle Class

Table 3.11 shows the details of the different steps of cleaning processing stage.

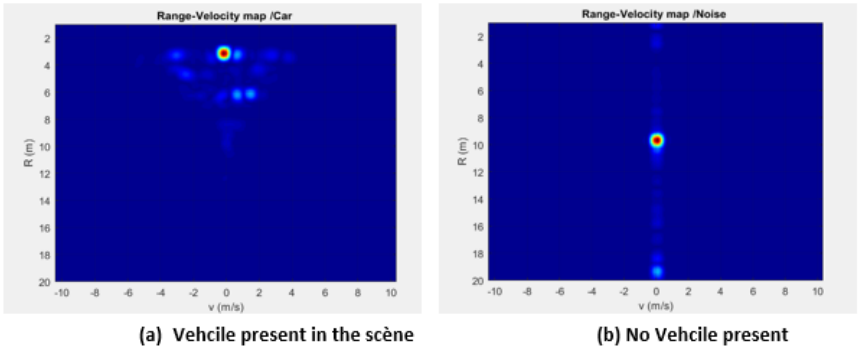


Figure 3.11: Range Doppler Map for Vehicle Class Measured Dataset

Table 3.8 shows the repartition of useful sequences containing vehicle Radar signature.

Measurement files	Lengths of the sequence (frame)	Useful sequence		
		Nb_useful	ID_start	ID_Stop
« file_1.mat »	150	18	80	97
« file_2.mat »	150	14	107	120
« file_3.mat »»	250	25	165	189
« file_4.mat »	250	20	165	185
4 files	800	78		

Table 3.8: Repartition of Useful Sequences for Vehicle Class

Figure 3.12 illustrates the result for the Vehicle Class, the red signal corresponds to all the frames concatenated before the cleaning processing and the one in blue corresponds to useful frames after the processing.

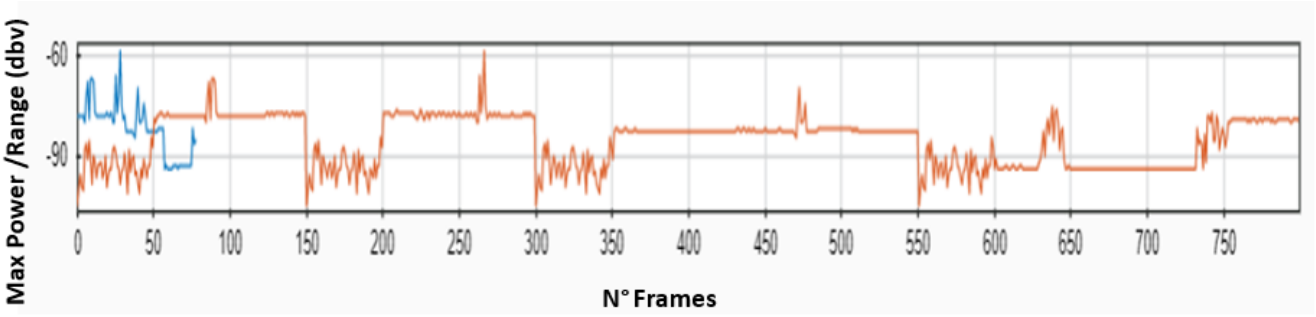
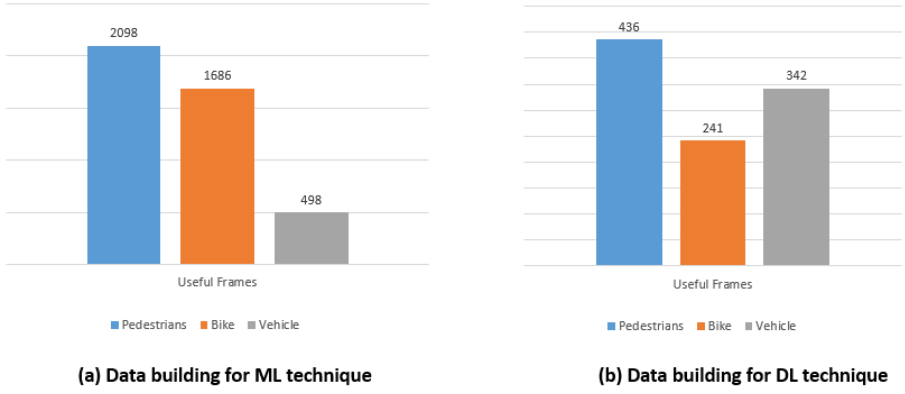


Figure 3.12: Reflected Power Repartition of Radar Frames before and after Cleaning Processing for Vehicle Class

In the same way, we clean the sequences recoded in the uncontrolled environment. As result, we get 4282 useful frames from the all data recorded for ML technique and 1019 useful frames for DL technique distributed as represented in figure 3.13.



(a) Data building for ML technique

(b) Data building for DL technique

Figure 3.13: Repartition of Useful Frames after Data Cleaning for the three Classes and ML & DL Techniques

3.3.2 Data Set Format

One mandatory part to elaborate database for RADAR Target recognition is the process of labeling the recorded data. Hence, we are facing a supervised learning problem since any error or inaccuracy affects negatively the database's quality and the overall performance of the predictive model. Under those circumstances, we operated as follow:

- First, we collected all radar target information (data + video)

- Second, to get a high-quality dataset, we labeled our data as finely as possible.

As a result, we recorded all scenarios of the different targets with the time shown in the video in milliseconds as shown in the figure3.14. Also, it has been ensured that the time shown in the video is synchronized with the radar data recorded.

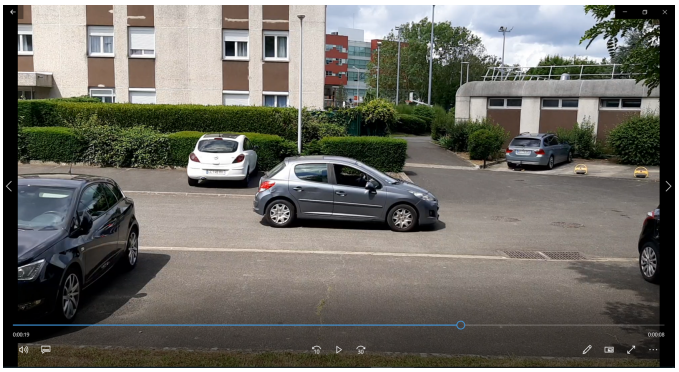


Figure 3.14: Video recording labeled with real time

3.3.3 Target Classes Data Set Analysis

In this sub-section, we will discuss the size of data set and distribution of frame between the three classes. As we can see in the figure 3.15 the data is highly imbalanced between pedestrians, bike and vehicle classes for the two datasets created for ML and DL techniques.

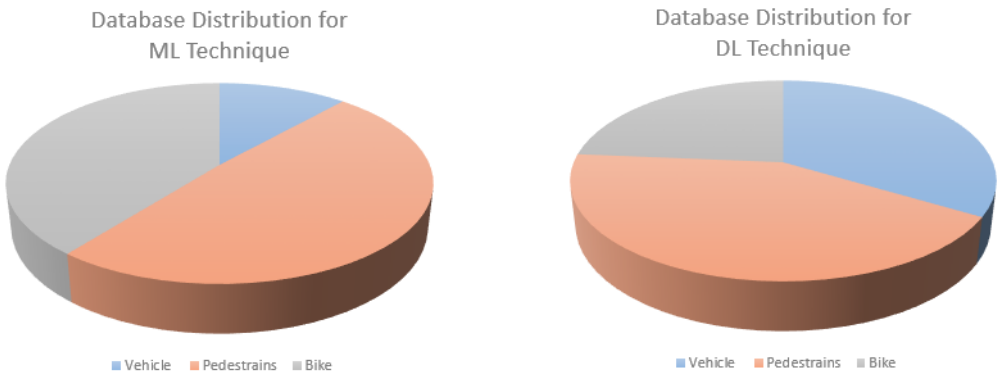


Figure 3.15: Database distribution for ML and DL Techniques

In that case, we propose two techniques:

- The first technique is one of the simplest ways to address the class imbalance. It simply provides a weight for each class which places more emphasis on the minority classes such that the end result is a classifier which can learn equally from all classes.

- The second technique is over and under sampling. Under sampling means we will select only some of the data from the majority class by only using as many examples as the minority class has. However, with this technique we will be wasting a huge amount of data. Also, the sample chosen by under sampling may be a biased sample. And it will not be an accurate representative of the population.

Thus, we decided to use the first technique to handle our imbalanced dataset. We specified class weights so they are inversely proportional to the size of the class.

$$weight(C_i) = \frac{numoframes}{numofclass \times numoframesinclass(C_i)}$$

3.4 Data Set Pre-processing

3.4.1 Data Set Pre-processing Objectives

The main objectif of this operation is to process data after each stage of the Radar Detection processing.The figure 3.16 illustrates the different bloc of signal processing. The following table indicates the data size at the output of each radar detection stage. This step is very important for the next step “feature extraction”:

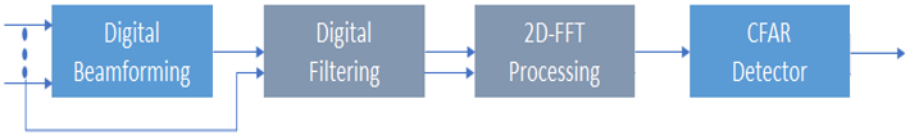


Figure 3.16: Radar Target Detection Signal Processing Stages

This study is used for selecting the input of AI classifiers. for ML techniques, features are extracted from different stages in the signal processing chain. As for DL technique, we chose to extract the doppler-range matrix as input to the classifier. Table 3.9 describes input and output signals for each detection processing stage.

Radar Detection Processing Stage	Input Data		Output Data	
	Input signals	Dimension	Output signals	Dimension
Digital Beamforming	Amplitude of the signal in time domain: V(**LSB**) : DAC output voltage	4 Power Signals in time-domain (1D dimension)	Amplitude of the signal in time domain: V(LSB)	1 Power Signal in time-domain (1D dimension)
Digital Filtering +Reshape	Amplitude of the signal in time domain: V(LSB)	1 Power Signal time-domain (1D dimension)	Amplitude of the signal in time domain: V(LSB)	1 Power Signal in time-domain (2D dimension)
2D FFT Processing	Amplitude of the signal in time domain: V(LSB)	1 Power Signal in time-domain (2D dimension)	Power Spectral Density (PSD)	1 PSD (2D dimension)
OS-CFAR	Power Spectral Density (PSD)	1 PSD (2D dimension)	Power Spectral Density (PSD)	1 PSD (2D dimension)

Table 3.9: Input and Output Signals Definition for the Radar Target Detection Processing Stages

3.4.2 Features Definition

Several studies have addressed the Radar Target Recognition (RTR) using conventional machine learning techniques for extracting features which are discussed in the following sections. The following table summarizes the most significant features used in some related works.

- Maximum Power per Range
- Variance of Power
- Mean Power Amplitude
- Std Power Amplitude
- Maximum Speed doppler
- Mean Speed doppler

- Variance Speed doppler
- Target speed
- Target size
- Extension in range
- Area (Number of detected point in Range-Doppler map)

Each feature constitutes a function of one or several data types contained within each radar detection. Any feature needs to be simple and significant in order to provide valuable information from many sources rather than much information from few sources.

Table 3.10 shows features structured in a CSV file.

	PowerperR	Max_vitesse_doppler	...	Target_speed	Class
0	-6.779965	2.096273	...	45.839000	car
1	-4.723249	-16.420807	...	39.597437	car
2	-6.625124	-12.228261	...	32.713345	car
3	-7.856965	-8.734472	...	49.220674	car
4	-7.966698	-7.686335	...	65.492421	car
5	-8.068174	-7.686335	...	38.141790	car
6	-8.292046	-5.240683	...	77.364778	car
7	-5.649163	-15.722050	...	57.283904	car
8	-8.605950	-3.843168	...	43.230036	car
9	-8.277398	-4.192547	...	44.662982	car
10	-8.402903	1.746894	...	62.115925	car
11	-8.452542	1.746894	...	38.561445	car
12	-8.081417	-7.336957	...	55.520191	car
13	-8.608676	-5.939441	...	37.979939	car
14	-8.323443	-2.445652	...	77.750147	car

Table 3.10: Results of Features Extracted

Figure 3.17 gives an idea about the distribution of the 11 features extracted.

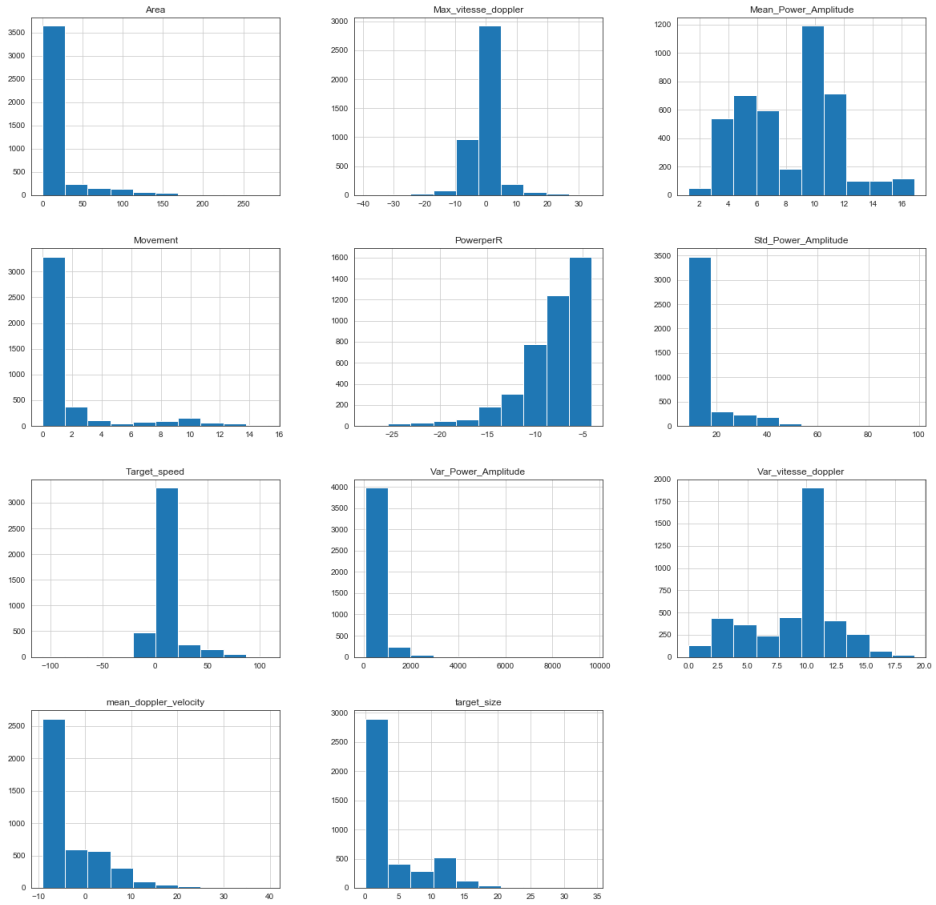


Figure 3.17: Features extracted histogram

According to the figure 3.17,we interpret that the distribution of features are not balanced.For example, we take the feature Max_Speed Doppler the majority are concentrated between 0 and 7 which corresponds to the Pedestrians Speed.

3.4.3 Features Selection

Feature selection is not only used to reduce data dimension but also to avoid irrelevant features that can affect badly the model. It is essential to notify that the processing of selected relevant features should be repeated for each model because each model has its most important features.

- **SelectKbest Features :**

  It consists in choosing those features that have the strongest relationship with the output

variable. To implement this technique, we have used the SelectKBest algorithm from the scikit-learn library Package [3].

Figure 3.18 shows the result of this selecting algorithm

	features	Score
0	PowerperR	249.821051
1	Max_vitesse_doppler	112.027501
2	Var_Power_Amplitude	633539.881899
3	Mean_Power_Amplitude	957.282368
4	Std_Power_Amplitude	2740.210014
5	Var_vitesse_doppler	11.425867
6	mean_doppler_velocity	6.357705
7	Movement	140.310750
8	Target_speed	224.365065
9	Area	1103.436238
10	target_size	348.793455

Figure 3.18: Database Features Scores Results with SelectKBest

According to this method , we can interpret that features extracted in time domain have the higher score. However, in the situation of multicibles are presence in the scene,these features are less significant.

- **Feature importance :**

This step aims to find the optimal number of the most important features. The Extra Tree Classier [4] is used for giving a score for each feature of the data, the higher the score the more important or relevant is the feature towards our output variable.

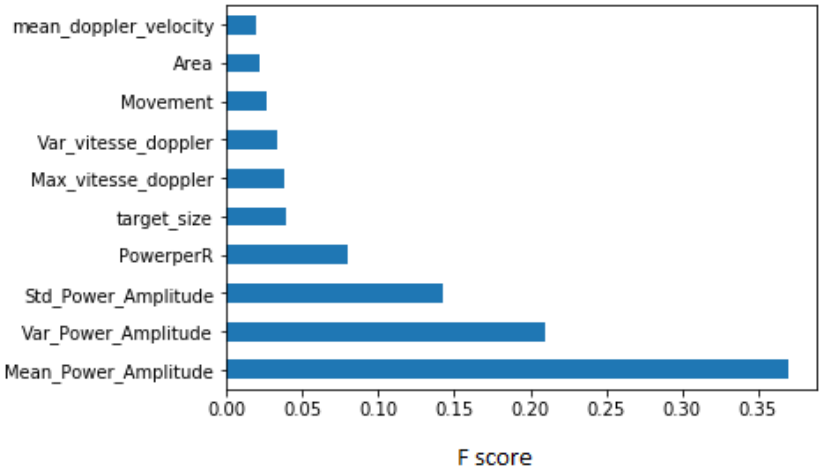


Figure 3.19: Database Features Scores Results using the Extra Tree Classier

As we can see from figure 3.20 that the first 6 proposed features are the most important features.

- **Pearson correlation:**

  This method looks at how well correlated two sets of data are.That is, how much does a variation in one data set affect the variation in another. We might want to use this as if there is a high correlation between one feature and the class of the data we are using then this will most likely be good at separating the data and will be useful for classification purposes.

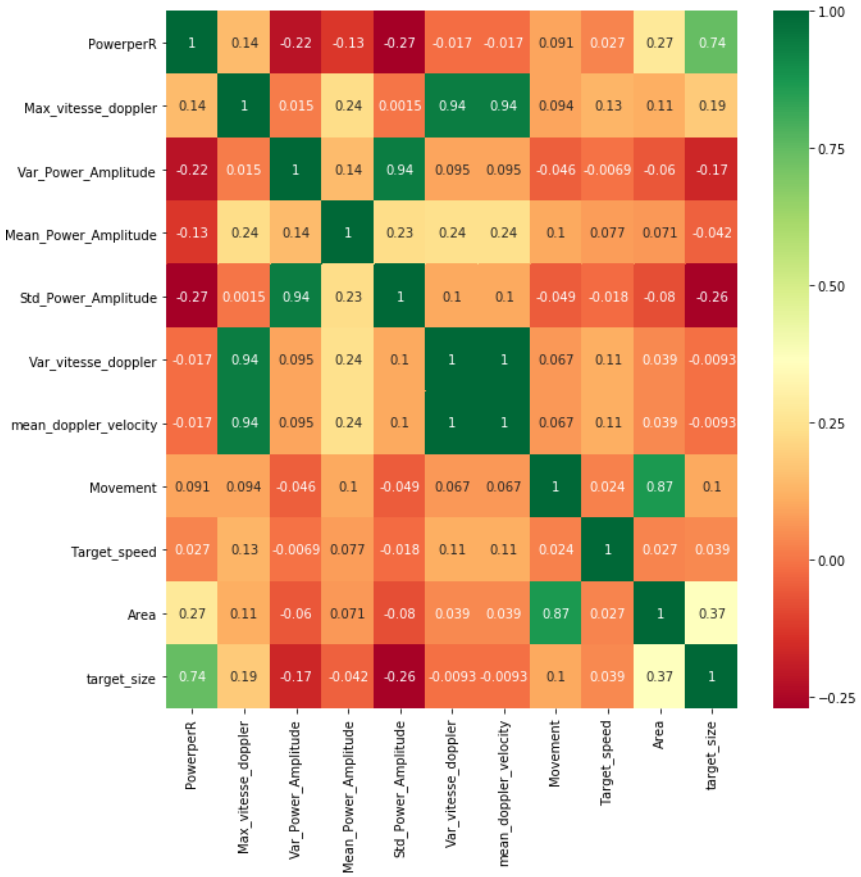


Figure 3.20: Correlation heatmap for the ML Dataset

By comparing the correlation between the features, we can easily see that those features who have a correlation higher than 0.9 should be removed.In our situation, We have five pairs of features that are strongly correlated (Colored in green).

- **Recursive Feature Elimination:**

  The RFE algorithm is used to select the top K features according to the model by recursively considering smaller and smaller sets of features.

```
In [341]: feature_selector_cv.n_features_
Out[341]: 4

In [342]: selected_features
Out[342]: ['Mean_Power_Amplitude', 'Std_Power_Amplitude', 'target_size', 'Target_speed']
```

Figure 3.21: Features Selection Results with RFE Algorithm

By applying this algorithm the number k obtained is equal to 4 and the selected features are :target_size, target_speed, Std_power amplitude,Mean doppler Speed.This algorithm identifies the features needed to fit the model with less complexity.

- **Feature reduction :**

  In order to increase the computational efficiency while retaining most of the information, the desired goal here is to reduce the dimensions of a K-dimensional data-set by projecting it onto a N-dimensional subspace (where N < K) using PCA with Scikit-learn [5].

  - Standardizing : since PCA performs best with a normalized feature set. The standard scalar normalization is used to normalize the feature set.
  - Applying PCA : Find the size of N (number of PC) that represents the data well.A useful measure is the so called explained variance for each PC), which can be calculated using a PCA's class named "explained variance ratio ".
  - Fit and transform : call the fit and then transform methods by passing the feature set to these methods. The transform method returns the specied number of PC.

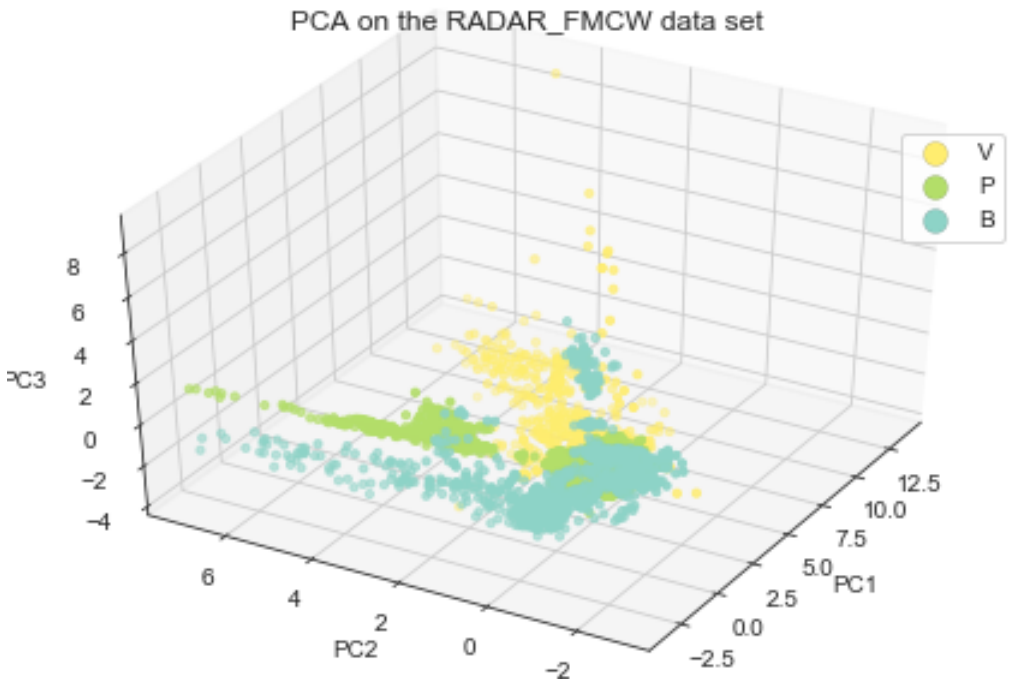


Figure 3.22: PCA Resut

As we can see from Figure 3.22 ,this step allows to have component more or less separable to the naked eye. we can interpret that this method makes it possible to reduce the dataset size but also to limit the correlation between the features by transforming the database into another space.

3.5 Conclusion

Through this chapter we have been interested to database creation with different steps. For the purpose of applying ML and DL techniques we prepared 2 two data sets. The first one is based on signal processing techniques to extract the most relevant features that will be fed into the ML and ANN classifier. We started by describing the test scenarios, gathering the data, cleaning data, applying different data engineering steps until we get CSV file that contains the 4282 frames. The second data set is based on 2D-fft from which we gathered the RD matrix that will be fed into the CNN model. In this stage, the features extraction is done automatically via convolution layers.

CHAPTER 4

Radar Target Classification Solution Implementation

4.1 Introduction

After presenting in the previous chapter the measured data preprocessing, the content of this chapter concerns the description of the models design for both machine learning and deep learning techniques. The first section of this chapter will present the ML-based models development and evaluation metrics. The second section will illustrate the two DL-Models for CNN and ANN approaches. In the last session, we define field test scenarios in different environments and we present test results of designed models and evaluate the robustness of the classifier.

4.2 Machine Learning-based Classification Solution

4.2.1 ML-based Models Development

a) Input pre-processing

- Encode Labels: encode labels using LabelEncoder function available in Scikit-learn package, and transform non-numerical labels to numerical labels: Encode labels with value between 0 and n_classes-1.
- Data Scaling : normalize the data set and translates each feature individually such that it is in the given range [0..1]. using the same package Scikit-learn.

b) Data Splitting

Split data into training and test Set using train_test_split function from Scikit-learn package. It is necessary to pass through this step to evaluate the performances of the models.



Figure 4.1: Data Splitting for both ML and DL Datasets

c) Model fitting

The classier performances are evaluated in terms of classification accuracy and classification accuracies per class. For this purpose, we are using 2 different validation protocols the k-fold and the Leave One Subject Out cross-validation protocols in this stage of processing.

c.1) K-fold cross validation process:

The procedure has a unique parameter called k which refers to the number of folds in which the data will be split into. As such, the procedure is often called cross-validation of k-folds. When a specic value is chosen, it can be used in the model reference, such as cross-validation multiplied by 10. It is a popular method because of its advantages such as its simplicity to understand and its ability to return results in a less biased or less optimistic estimate of the model skill than

other methods, such as a simple train/test split. Also the variance of the resulting estimate is reduced as k increases. However it limitation reside in the fact that it allow some of data of a specic subject to be used in both training and testing in the same iteration.

c.2) Leave-one-subject-out-cross-validation:

Leave-one-subject-out cross-validation is a particular case of K-fold cross-validation where a subject can be seen as a fold. Hence, the number of subjects determine the number of folds. It involves using the observations of one subject as the validation set and the remaining observations as the training set. Hence, with this technique we guaranty that we do not have subject bias.

The data is ready now to be processed by the ML models. The next step is to train multi ML model and then select the most performance one in term of accuracy.

d) Saving the model

At this step , we have saved the model in the "'file .sav"' using the pickle package in python.to can predicted the label of the new frames in real-time. And then, we can implemente the model on the hardware trarget.

4.2.2 ML-based Models Accuracy Analysis

In order to select the best algorithm that suit with the dataset created we have tried 9 ML models:

- Gaussian Naive Bayes [6]
- k-nearest neighbors[7]
- Naive Bayes classifier for multivariate Bernoulli models[8]
- Logistic Regression[9]
- Linear Support Vector Classification[10]
- Nu-Support Vector Classification[11]
- Decision Tree Classifier[12]
- Support Vector Machine [Appendix A]

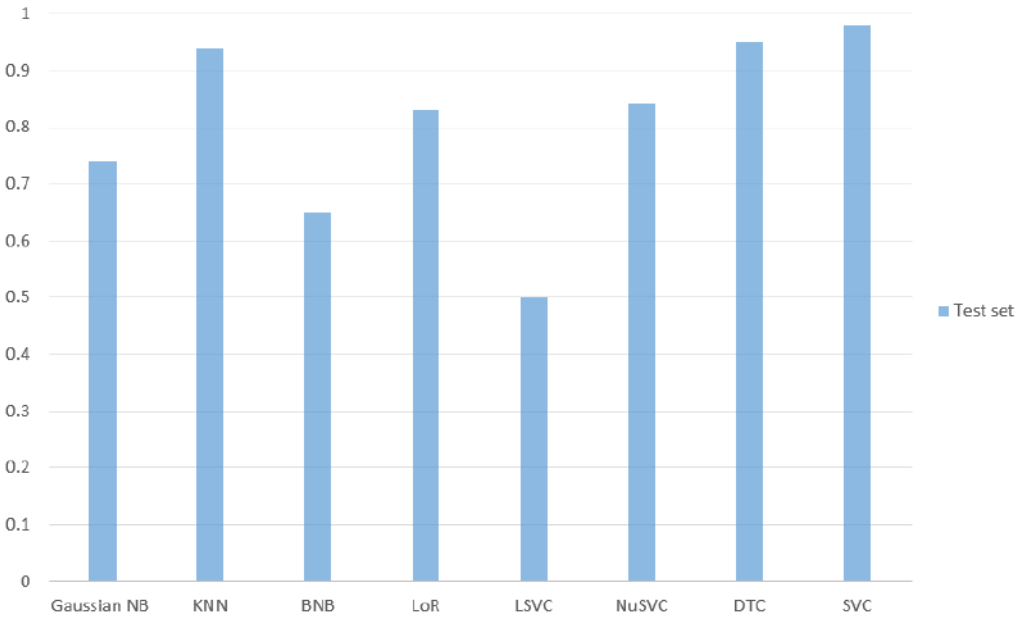


Figure 4.2: Model Accuracy for ML-based Models

As can be seen in figure 4.2, the SVM shows the best accuracy equal to 0.99 compared to the other algorithms . Thus, this model has been selected to optimize target's classification.

4.2.3 SVM-based Model Optimization

The radial basis function (RBF) was chosen for the support vector machine kernel. The RBF kernel is well-behaved mathematically, uses only a single free parameter γ , and performs well on radar target recognition. When using the RBF kernel, the SVM designer must provide appropriate values for the cost parameter C and the RBF kernel parameter γ .

The results of a grid search over the cost parameter C and kernel parameter $\gamma$are shown in Table 4.1.

Model 1	SVM (2 component)	Kernel ="rbf"	C= 100; gamma =1
Model 2	SVM (3 component)	Kernel ="rbf"	C= 100; gamma =0.1
Model 3	SVM (11 component)	Kernel ="rbf"	C= 1000; gamma =10e-5

Table 4.1: Results of SVM Model Optimization

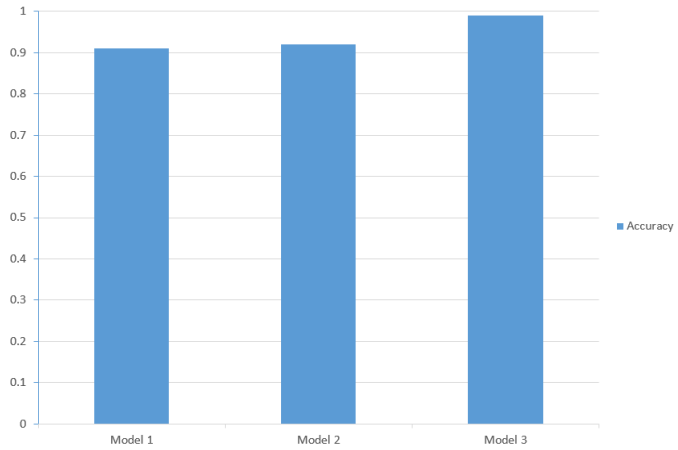


Figure 4.3: Model Accuracy for the 3 Designed SVM Models

	precision	recall	f1-score	support
vehicle	0.87	0.92	0.89	319
pedestrians	0.95	0.91	0.93	437
bike	0.97	0.96	0.97	101
micro avg	0.92	0.92	0.92	857
macro avg	0.93	0.93	0.93	857
weighted avg	0.92	0.92	0.92	857

(a) Classification report of the SVM with 2 Component

	precision	recall	f1-score	support
vehicle	0.98	0.76	0.86	319
pedestrians	0.85	0.99	0.92	437
bike	0.98	0.98	0.98	101
micro avg	0.90	0.90	0.90	857
macro avg	0.94	0.91	0.92	857
weighted avg	0.91	0.90	0.90	857

(b) Classification report of the SVM with 3 Component

	precision	recall	f1-score	support
vehicle	0.99	1.00	1.00	319
pedestrians	1.00	1.00	1.00	437
bike	1.00	0.98	0.99	101
micro avg	1.00	1.00	1.00	857
macro avg	1.00	0.99	1.00	857
weighted avg	1.00	1.00	1.00	857

(c) Classification report of the SVM with 11 Component

Figure 4.4: Classification Report for the 3 Designed SVM Models

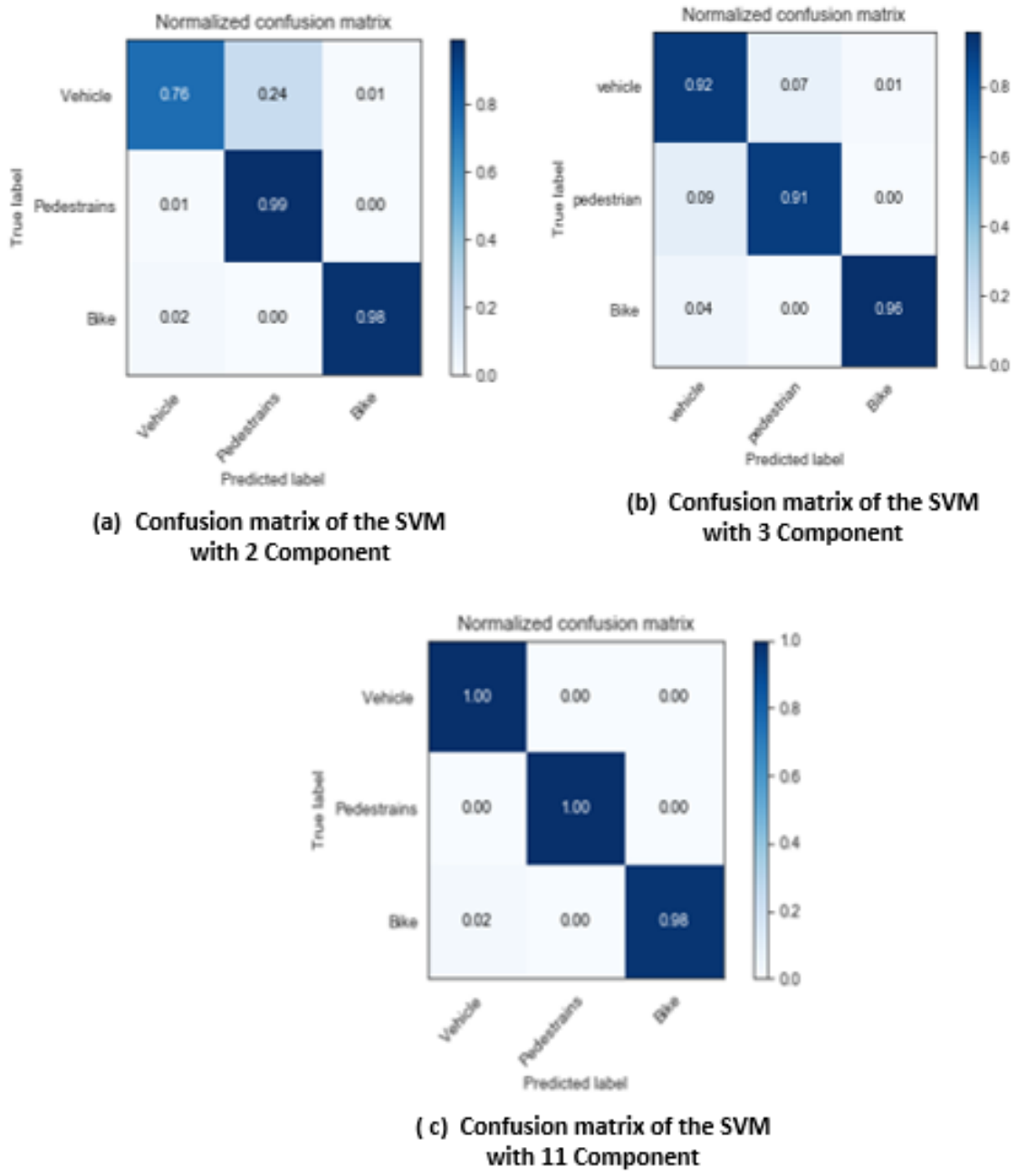


Figure 4.5: Confusion Matrix for the 3 Designed SVM Models

By analyzing the ROC curves for the three designed SVM models, we conclude that the 3 models succeed in distinguishing between the three classes (pedestrian, bike, vehicle) but the most robust one is the model with 11 components. Figure 4.6 illustrates the AUC-ROC for the 3 SVM models.

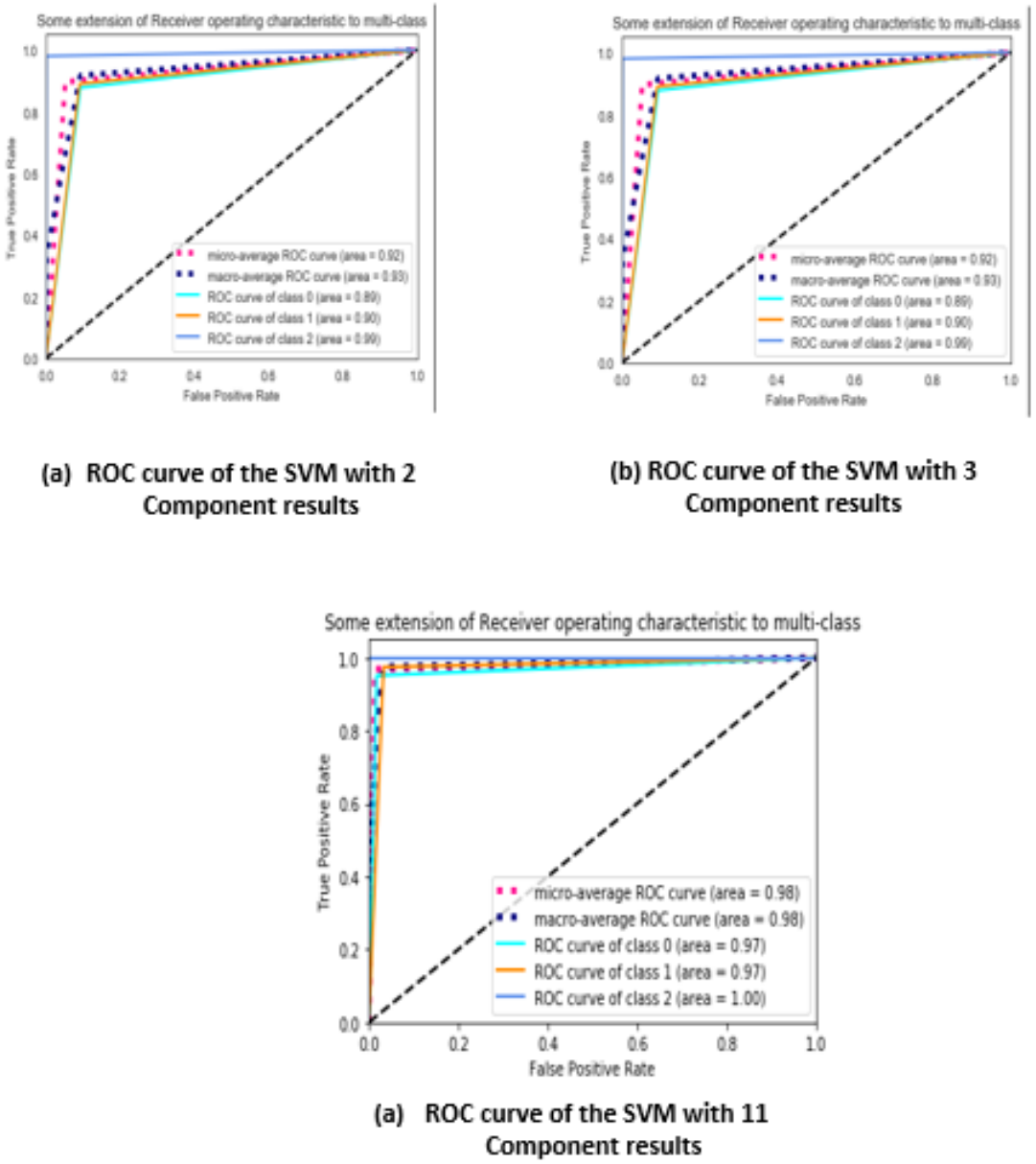


Figure 4.6: AUC-ROC Curves for the Designed SVM Models

4.3 Deep Learning-based Classification Solution

4.3.1 CNN-based Model Development

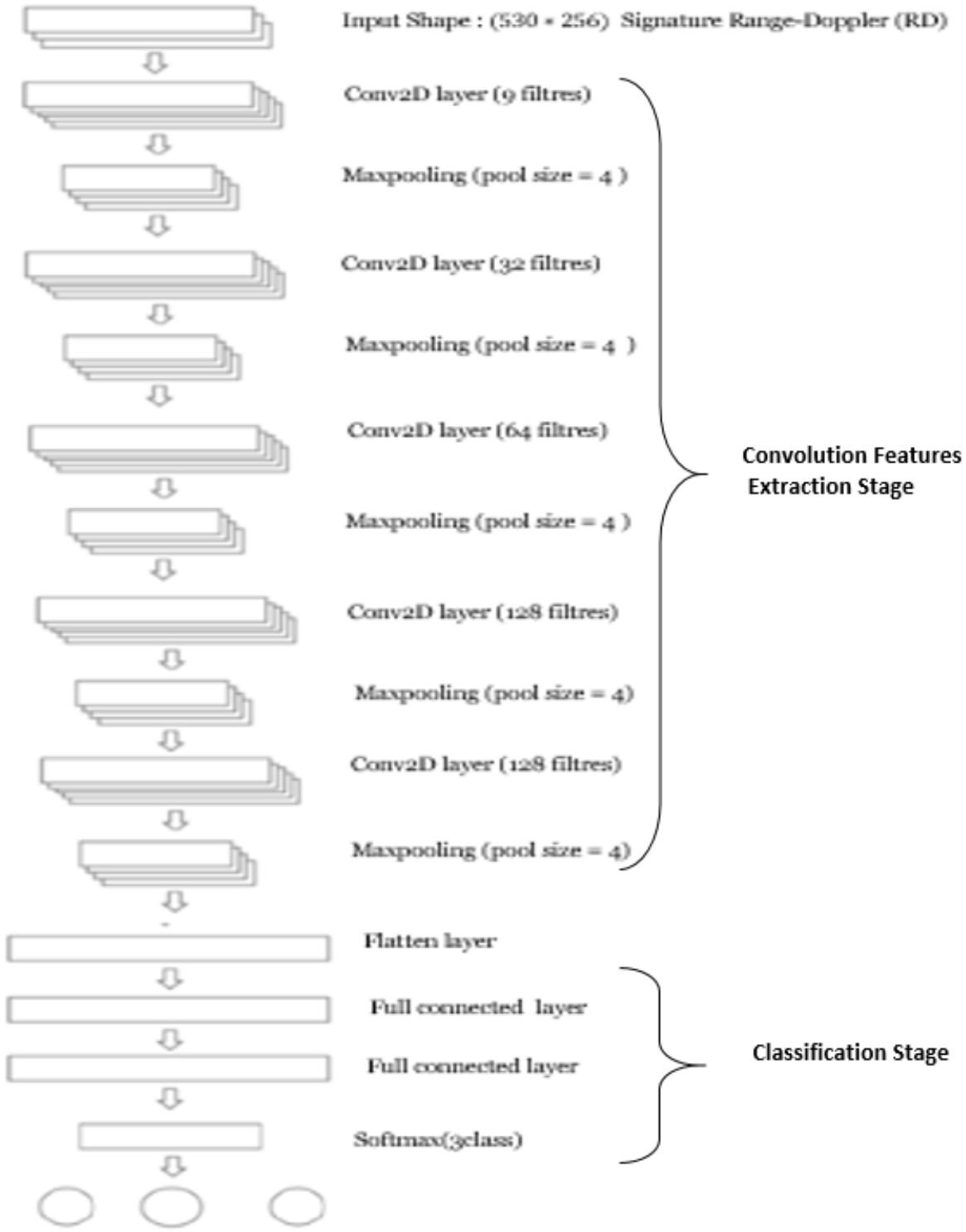


Figure 4.7: CNN-based Classification Processing Stages

The overall network architecture is composed of an input stage, a feature extraction stage with five convolution layer blocks, and an output provision classification stage. The input stage received the preprocessed dataset. The feature extraction stage followed a quintuplet ternary convolution block structure including convolution, pooling, and activation functions. The last classification stage consisted of a fully connected layer, and softmax operator. The final network outputs came from the softmax operation results.

- **Convolution stage**

  (i) Input layer: In a CNN, a layer can be represented by the general expression function $y = f(x)$where both the input x and output y are tensors. The network input is a Height(H) Width(W) Dimension(D) sized tensor data. In this study, the input was the Range doppler signature $H = NfftRange$, $W = Nfftvelocity$, and $D = numberofframes$. this input size H W D was equal to $530 * 256 * 1$.

  (ii)Convolution layer: The convolution layer computed the input x convolution using filters f in order to detect local conjunctions of features from the previous layer and mapping their appearance to a feature map. However, in this study we considered one type of convolution layers with bi-dimensional kernels because CNN with 2D kernel is able to capture spatial dependency over range as well as local dependency over velocity at one frame. This method is useful for multi target scenarios where we have more than target to extract their features.

  (iii)Pooling layer: Pooling layers are used to identify the most important features and compresses layer sizes by sub sampling. In this study, a max pooling method is used after a convolution stage in order to reduce the computational requirements progressively through the network as well as minimizing the likelihood of overfitting.

  (iv)Activation function: An activation function followed the max pooling step in the convolution block. A rectified linear unit (ReLU), was selected for this function to mitigate the vanishing gradient problem and reduce learning time. Any negative output values remaining after the max pooling layer were set to zero.

  (v)Repeated convolution blocks: five convolution blocks were used,combining the steps of convolution, max pooling,and ReLU. Figure 4.7 shows the quintuple convolution block structure, the filter sizes for all the three blocks.

- **Classification stage**

(i)Flatten layer:A flatten operation on a tensor reshapes the tensor to have the shape that is equal to the number of elements contained in tensor.

(ii) Fully connected layer: The ReLU results of each layer, as with all layer outputs, were connected to the weighted nodes of the next layer.

(iii) Softmax layer: The softmax function was applied across feature channels to consolidate network results. This normalization method was applied to make that the sum of output probability distributions equal to 1.

After designing the model using the keras package, the input-pre-processing, encode label, data spliting steps are repeated in this DL technique with RD dataset in order to prepare data as input of the CNN-2D classifier. The next step for Model fitting consists on training the model with the training set and validate it with the validation set which is the test set in this case. For that, Adam is used as optimizer with a learning rate lr equal to 0.0005, a batch size of 50 records and 40 epochs as a hyper-parameter. In order to avoid over-fitting, we used dropout layer with different rate between convolution blocks and dense layer

Dropout layer: the dropout layer was imported to prevent overfitting, which may occur if a node in one layer excessively influences the subsequent layer. The dropout step dropped randomly selected nodes in fully connected layers with a set turn-on rate. Regularizing dropouts thinned the ongoing training network and created an ensemble effect within the thinned network . This step could be performed in every fully connected layer except the final one for output.

Layer (type)	Output Shape	Param #
conv2d_13 (Conv2D)	(None, 528, 254, 32)	320
max_pooling2d_13 (MaxPooling	(None, 264, 127, 32)	0
conv2d_14 (Conv2D)	(None, 262, 125, 64)	18496
max_pooling2d_14 (MaxPooling	(None, 131, 62, 64)	0
conv2d_15 (Conv2D)	(None, 129, 60, 128)	73856
max_pooling2d_15 (MaxPooling	(None, 64, 30, 128)	0
conv2d_16 (Conv2D)	(None, 62, 28, 128)	147584
max_pooling2d_16 (MaxPooling	(None, 31, 14, 128)	0
flatten_4 (Flatten)	(None, 55552)	0
dense_7 (Dense)	(None, 512)	28443136
dense_8 (Dense)	(None, 3)	1539

Total params: 28,684,931
Trainable params: 28,684,931
Non-trainable params: 0

Figure 4.8: CNN-2D Model Design Parameters

As we notice in figure 4.9, the proposed model has reached with a training accuracy of 75% despite the limitation of the dataset size. In fact, the model accuracy has increased from 44% to 75% thanks to the proposed model parameters adjustment.

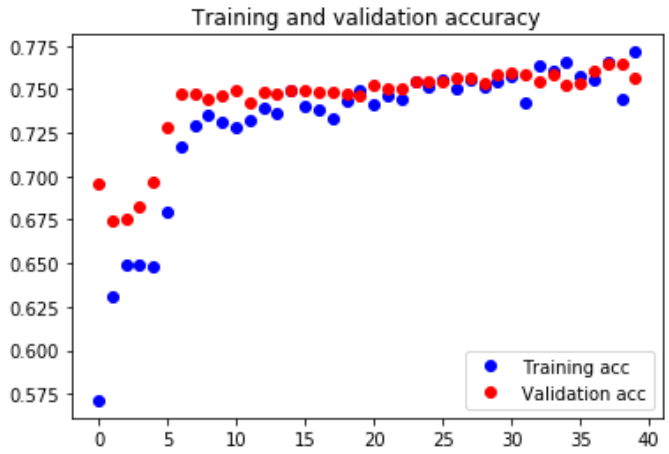


Figure 4.9: Training and Validation Accuracy of the CNN-based Model

As illustrated in figure 4.10, the training and validation loss decreases to a point of stability with a minimal gap between the two final loss values. This means that the model is well generalized and learned with a good fit.

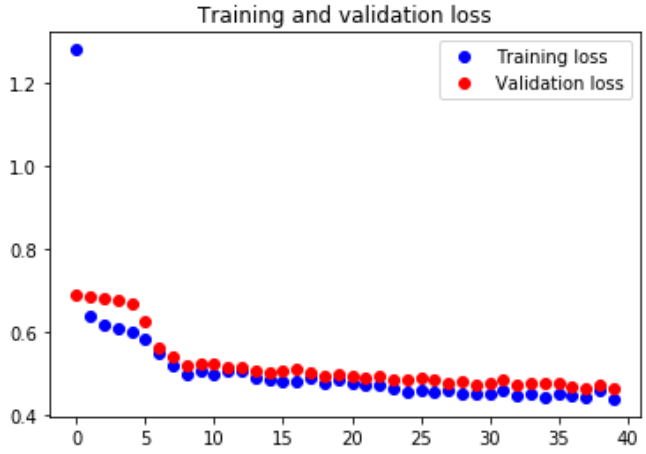


Figure 4.10: Training and Validation Loss of the CNN-based Model

4.3.2 ANN-based Model Development

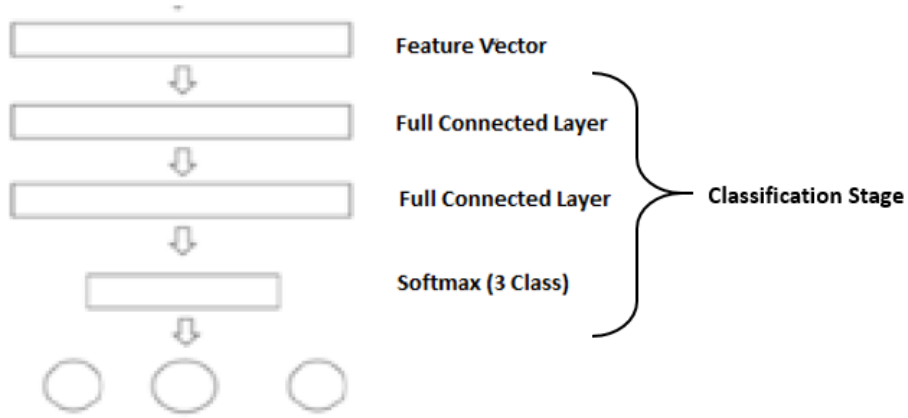


Figure 4.11: ANN-based Classification Processing Stages

The overall network architecture is composed of two stages. the first one is the input stage and the second one is the output provision classification stage. The input stage received the extracted features preprocessed. The classification stage consisted of a fully connected layer, and softmax operator. The final network outputs came from the softmax operation results. The final network outputs came from the softmax operation results. after we prepared the model. we fed the data into the classifier following the same step as the previous classifiers (scaling input, spiting dataset, test validation ...)

It is clear that there is no problem of confusion to identify all the classes as shown the figure 4.12 and 4.13.

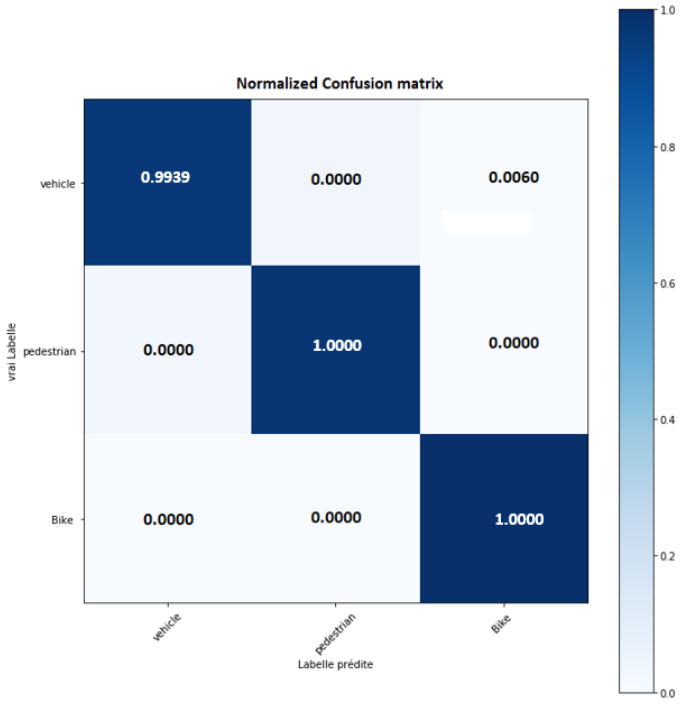


Figure 4.12: Normalized Confusion Matrix for ANN model

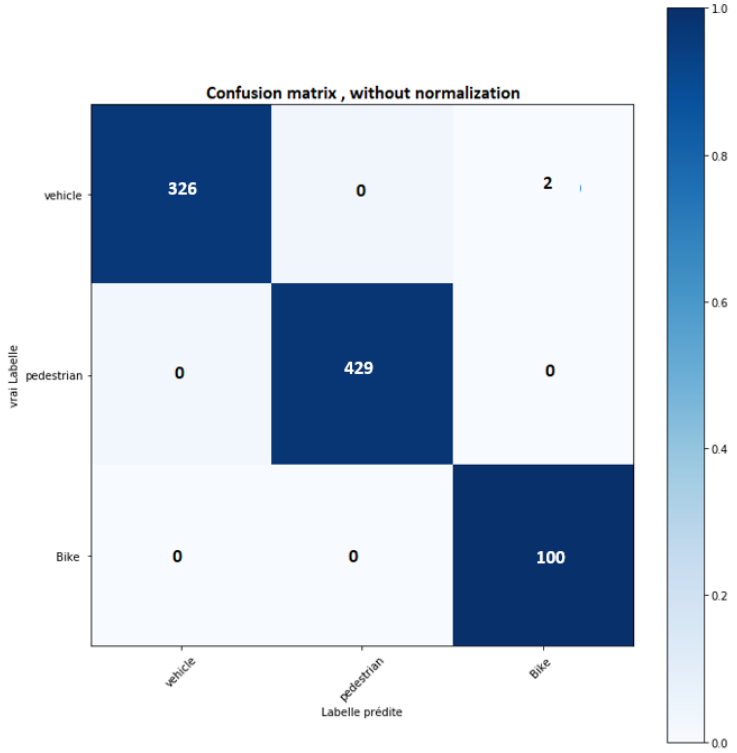


Figure 4.13: Confusion Matrix without Normalization for ANN model

The accuracy of the model is in general good for predicting the whole classes, with a test accuracy around to 95%. Figure 4.14 shows the classification report for the test data.

```
Baseline: 91.19% (12.39%)
              precision    recall  f1-score   support

           0       1.00      0.99      1.00       328
           1       1.00      1.00      1.00       429
           2       0.98      1.00      0.99       100

   micro avg       1.00      1.00      1.00       857
   macro avg       0.99      1.00      1.00       857
weighted avg       1.00      1.00      1.00       857
```

Figure 4.14: Classification report for ANN

4.4 Field Test Results of Target Classification Solutions

In this section we will evaluate for new field test scenarios the road targets classification performances for the designed 3 SVM models and the ANN-based model that gave an accuracy higher than 90%. The classification accuracy is measured by dividing the number of predicted correct frames by the total number of frames.

4.4.1 Classification Results for Field Test Scenario 1

Test location	Urban road
Time	Day time
Radar characteristics	6 m width
Radar mounting	Offset = 6 m , Height =1.5 m , Orientation = horizontal
Number of Targets	1
Class of Targets	Vehicle

Table 4.2: Test Scenario Definition for Vehicle Target Class



Figure 4.15: Classification Test Scenario Result for Vehicle Target Class

Model	Precision
SVM with 2 component	98.5%
SVM with 3 component	98%
SVM with 11 component	92%
ANN	100%

Table 4.3: Classification Accuracy Results for Test Scenario 1

As we can see for this scenario the highest precision is that of the ANN classifier with100% for pedestrians target to discriminate vehicle target against all existing interefernats.

4.4.2 Classification Results for Field Test Scenario 2

Test location	Outdoor, Campus parking
Time	Day time
Radar characteristics	8 m width
Radar mounting	Offset = 4m , Height =1 m , Orientation = horizontal
Number of Targets	1
Class of Targets	Pedestrian

Table 4.4: Test Scenarios Definition for Pedestrian Target Class



Figure 4.16: Classification Test Result for Pedestrian Target Class

Model	Precision
SVM with 2 component	89%
SVM with 3 component	80%
SVM with 11 component	81.25%
ANN	93%

Table 4.5: Classification Accuracy Results for Test Scenario 2

As we can see for this scenario the highest precision is that of the ANN classifier with 93% for pedestrian target to discriminate pedestrian target against all existing interefernats.

4.4.3 Classification Results for Field Test Scenario 3

Test location	basketball court
Time	Day time
Road characteristics	12 m width
Radar mounting	Offset = 4m , Height =2 m , Orientation = horizontal
Number of Targets	1
Class of Targets	Bike

Table 4.6: Test Scenario Definition for Bike Target Class

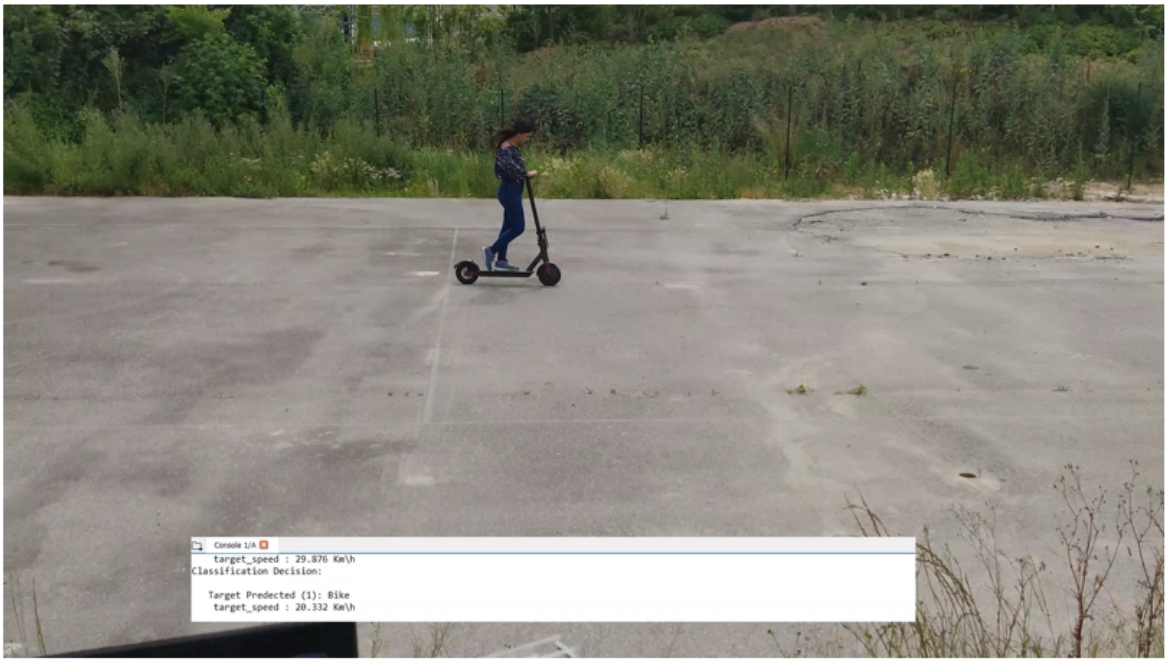


Figure 4.17: Classification Test Result for Bike Target Class

Model	Precision
SVM with 2 component	83.25%
SVM with 3 component	50%
SVM with 11 component	95%
ANN	98%

Table 4.7: Classification Accuracy Results for Test Scenario 3

As we can interpret for this scenario the highest precision is that of the ANN classifier with 98% for the Bike target to discriminate bike target against all existing interefernats.

4.4.4 Classification Results for field Test Scenario 4

Test location	Urban road
Time	Day time
Radar characteristics	6 m width
Radar mounting	Offset = 6 m , Height =1.5 m , Orientation = horizontal
Number of Targets	2
Class of Targets	Vehicle , Pedestrian

Table 4.8: Test Scenarios Definition for Multi-targets

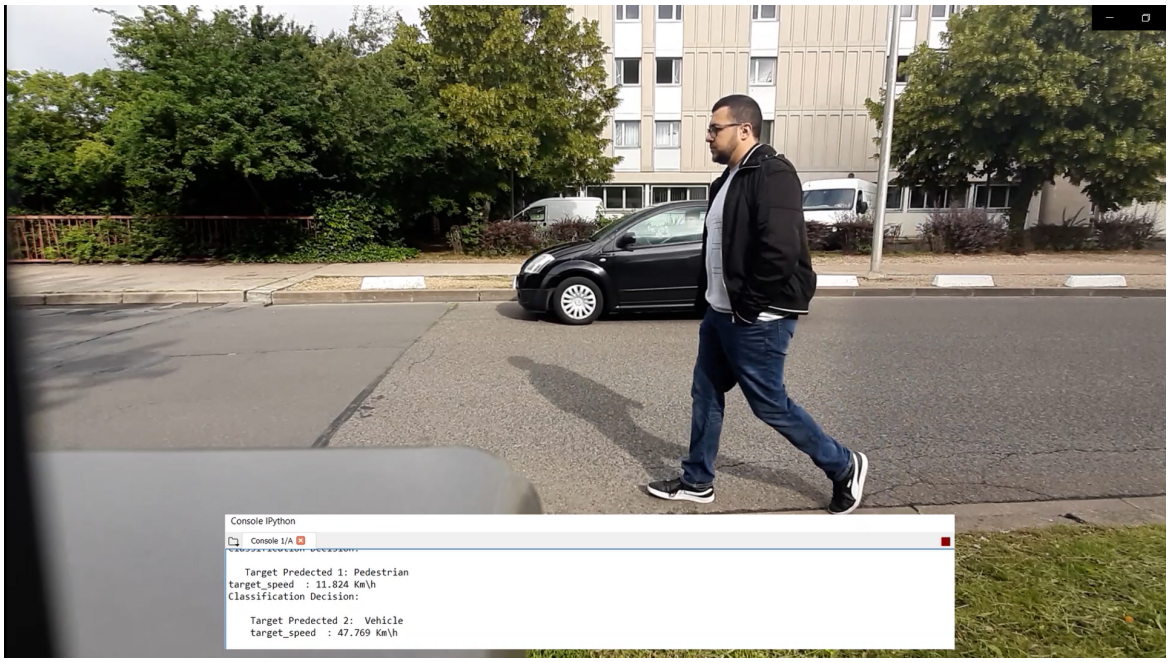


Figure 4.18: Classification Test Results for Multi-targets

Model	Precision(%)	
	Target 1 (pedestrain)	Target 2 (vehicle)
SVM with 2 component	95	80
SVM with 3 component	98	80
SVM with 11 component	81.25	70
ANN	94.25	90

Table 4.9: Classification Accuracy Results for Test Scenario 4

As we can see for this scenario the highest precision is that of the ANN classifierr with 94.25% for the 1st target and 90% for the 2nd target.

In the 4 scenarios, we can notice that the dna is the middle classifier because it gives the best precesion compared to others

4.5 Conclusion

In this chapter we focused in various classification techniques for radar monitoring application. We started by the ML technique then we tried different DL techniques. For the ML technique the SVM gave better performance regarding the others ML techniques by reaching an accuracy up than 92%. For the DL approach we tried the CNN and ANN classifiers. Due to some limitations in dataset size we can interpret that ANN model is more performant than the CNN. Finally, we exposed some test scenarios that allowed the validation of the selected model. In this step, we affirmed that the ANN model is well suited to the radar target classification in more critical scenarios.

General Conclusion

In this study, the classification of moving target for traffic monitoring application was performed. This study involved algorithm development, Dataset design, data collection and features engineering. The developed algorithms distinguished moving target into three different classes (vehicles, bikes, pedestrians).

We subsequently developed three classification models namely the SVM as a conventional machine learning technique, and two variants of DL methods. The first DL CNN-based technique is defined with bi-dimensional (2D) kernels used to extract local dependency over the range profile as well as the Doppler profile dependency over different frames captured using the radar. The second DL ANN-based technique have been developed by using data set defined .

The results obtained show that the ANN allowed the best classification accuracy (arround 97% with k-fold cross validation protocol) compared to the SVM (92%) and CNN (75%) classifiers.

This study represents some limitations such as the size of the data set was restricted to 1000 frames in CNN and 4000 frames for ML-techniques which is limited for the validation of the results. To do this, we suggest increasing the size of the database by adding the other scenarios for different classes to test and validate the different classifiers used in this study as well as we can include another sub-class for better performances.

Other lines of investigation might be interesting in the field of radar target classification by using other pre-processing data such as the micro-Doppler effect (STFT), The discrete-time real cepstrum (DCT).

In addition, one can invest in hierarchical model classifiers for different targets and include human activity classification based on radar recognition .We can also transform the problem of data labeling to classification based on unsupervised problem which we use k-mean algorithms to assign label to the frame. The detection algorithm can be done by investing in the field of

artificial intelligence and reasoning by analogy to the target detection in an image. Under those circumstances, we aim to use radar to build new concept of the smart road.

Appendix A : Support Vector Machine

The support vector machine (SVM) is a learning method rst introduced by Vladimir Vapnik in [13]. The method was further rened in [14] and [15], and has since become one of the most widely used machine learning methods.

The support vector machine is a maximum margin method that can be used for both classication and regression.

One of the benets of SVM is that it is formulated as an optimization problem instead of an iterative method such as the neural network. This is benecial since it provides a mathematical insight into the structure. Another advantage is that the training can be performed using optimization techniques such as quadratic programming [16].

Below, a linear support vector machine is shown. The name of the method is derived from the ”support vectors” - the data points that exist closest to the decision boundary, thus laying on the margin itself.

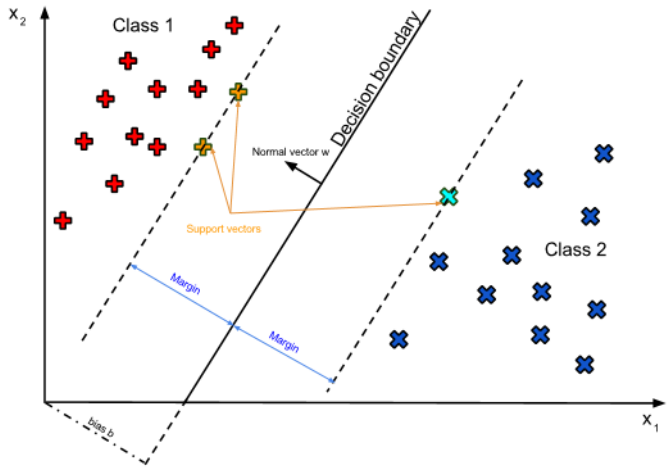


Figure A1: The support vector machine visualized in a two-dimensional feature space

In figure above, the concept of support vectors and margin are visualized. More details about the mathematical formulation of SVM are presented below.

Mathematical formulation and primal problem Since a linear decision boundary

takes the form of a hyperplane, it can be expressed as the set of points x that satises:

$$\omega \times x + b = 0 \tag{A1}$$

Here ω represents the normal vector of the hyperplane, while the bias b is the distance along ω to the origin. The margin is dened by two separate hyperplanes both parallel to the decision boundary:

$$\omega \times x + b = -1 \qquad \omega \times x + b = 1 \tag{A2}$$

The size of the margin can be expressed as the geometrical distance between the two planes of equation A2:

$$M_{size} = \frac{2}{||\omega||} \tag{A3}$$

The objective of support vector training is to maximize M_{size} while ensuring that no data points exist between the two planes of equation A2. In fact only data points of class 1 should exist beyond the rst hyperplane, and only points of class 2 beyond the other.

Mathematically, this condition is formulated as two inequality constraints.

For all data points x of the rst class:

$$\omega \times x + b \leq -1 \tag{A4}$$

For all data points x belonging to the second class:

$$\omega \times x + b \geq 1 \tag{A5}$$

Using the class label $y_i = \pm 1$ the two constraints A4 and A5 can be rewritten as a single inequality constraint:

For all N training data points $1iN$:

$$y_i \times (\omega \times x + b) \geq 1 \tag{A6}$$

From equation A3, it can be seen that in order to maximize the size of the margin, $||\omega||$ needs to be minimized.

Thus, the primal SVM optimization problem is stated as:

$$\begin{aligned} & \min_{\omega,b} \frac{1}{2}||\omega||^2 \\ subject \quad to \quad & y_i \times (\omega \times x + b) \geq 1 \\ & i = 1, ..., N \end{aligned}$$

Since the evaluation of $||\omega||$ requires a square root, the reformulation into $1\ \frac{1}{2} \times ||\omega||^2$ is made. This has the same minima and results in a problem suitable to be solved using quadratic programming techniques

Dual SVM formulation In equation A6, the primal formulation of the SVM optimization problem was given. However, when implementing support vector machines, it is more common to make use of the dual problem.

In [16], the dual formulation is written as:

$$\begin{aligned} & \min_{\boldsymbol{\alpha}} \frac{1}{2} \cdot \boldsymbol{\alpha} \cdot diag(\boldsymbol{y}) \cdot \boldsymbol{G} \cdot diag(\boldsymbol{y}) \cdot \boldsymbol{\alpha} - \boldsymbol{e} \cdot \boldsymbol{\alpha} \\ & subject\ to \quad \boldsymbol{\alpha} \cdot \boldsymbol{y} = 0 \\ & \alpha_i \geq 0 \qquad i = 1, \ldots, N. \end{aligned}$$

This formulation of the SVM problem is on a form readily solvable using convex optimization toolboxes. Here e is a vector containing only ones, while diag($\mathbf{y}$) is a diagonal matrix containing all class labels $y_i = \pm 1$. Also used is the Gram matrix of dot products $G \equiv x_i \times x_j$.

Solving the optimization problem (14) yields the vector of Lagrange multipliers α. These can in turn be used to nd the optimal separating hyperplane normal vector ω:

$$\hat{\boldsymbol{\omega}} = \sum_{i=1}^{N} (\alpha_i y_i \boldsymbol{x}_i)$$

Only a small subset of the training points xi will have corresponding $\alpha_i 6 = 0$. For these points, the inequality constraint in equation (12) will be an equality constraint:

$$y_i \times (\omega \times x + b) = 1 \tag{A7}$$

This means that they lie exactly on the edge of the margin; they are the support vectors [16].

Once the optimal hyperplane normal vector $\vec{\omega}$ is known, equation (15) can be used to obtain the hyperplane bias b:

$$b = y_i - \hat{\boldsymbol{\omega}} \cdot \boldsymbol{x}_i$$

In[16], the bias value used is a weighted average value of b over all support vectors:

$$\hat{b} = \sum_i \alpha_i (y_i - \hat{\boldsymbol{\omega}} \cdot \boldsymbol{x}_i) / \sum_i \alpha_i$$

When $\vec{\omega}$ and $\vec{b}$ are known, the SVM classier is constructed. The decision boundary is dened by equation (7) and thus any new datapoint x_n can be classied according to

$$class(\boldsymbol{x}_n) = \text{sgn}(\hat{\boldsymbol{\omega}} \cdot \boldsymbol{x}_n + \hat{b})$$

Kernels and soft margin The method on which SVM is based was rst published in 1963 in [13]. However, this method, as well as the formulation given in equation (13), is unable to handle classication problems unless they are linearly separable.

In practise, very few data sets can be separated this way, and for this reason the method was not widely used. In [14], a way to enable nonlinear support vector classication using kernels was presented.

If the feature data can be transformed to a feature space of higher dimensionality, the classes may be linearly separable in that space. A linear decision boundary that separates the points in the high-dimensional space will be non-linear in the original feature space, enabling separation of non-linear data sets.

A simple illustration of the transformation and resulting decision boundaries is given in gure 10. Mathematically, the transformation is described by the function . If x Rn then (x) RN, where $N > n$.

To enable advanced decision boundaries in the original space, N is usually a very high number. With the primal formulation of SVM, this can lead to computing problems since the resulting optimization problem will be extremely big. With the dual formulation however, the size of the optimization problem will not change [16].

What makes the method even more powerful is that there is no need to compute any transform (x) directly, due to what is called the kernel trick. Since the dual formulation of SVM (see equation (14)) only contains data points x in the form of dot products within the

Gram matrix, it is enough to compute the dot products of the points in the transformed space $(x_i)\Delta(x_j)$.

The kernel function K is a way to calculate these dot products implicitly, without ever computing the data point coordinates in the higher-dimensional space.

Thus in order to implement non-linear classication with SVM, the only thing needed is to replace the Gram matrix$G \equiv x_i \times x_j$ with the kernel function $K(xi, xj) \equiv (x_i) \times (x_j)$.

The function $K(x_i, x_j)$ can be chosen as any function that satises certain properties of an inner product. When implementing support vector machines however, it is common to use one of the following basic kernel functions [16]:

- Linear kernel (the Gram matrix)

$$K(\boldsymbol{x}_i, \boldsymbol{x}_j) = \boldsymbol{x}_i^T \boldsymbol{x}_j$$

- Sigmoid Kernel

$$K(\boldsymbol{x}_i, \boldsymbol{x}_j) = (\gamma \boldsymbol{x}_i^T \boldsymbol{x}_j + r)$$

- Polynomial Kernel

$$K(\boldsymbol{x}_i, \boldsymbol{x}_j) = (\gamma \boldsymbol{x}_i^T \boldsymbol{x}_j + r)^p$$

- Radial basis function (RBF kernel)

$$K(\boldsymbol{x}_i, \boldsymbol{x}_j) = e^{-\gamma \|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2}$$

Here γ, p and r are kernel parameters. A method to choose suitable values for these parameters is given in the section about model selection further below.

It should be noted that the usage of kernels may result in a slightly extended execution time in the prediction stage, since the Gram matrix only represents a dot product, while the dierent kernel functions may require additional computations.

If class separation is possible using the linear kernel, it should always be chosen, since this will minimize execution time and also reduce the risk of overtting.

Soft margin To reduce the risk of overtting, it is common in machine learning techniques to have what is called a regularization parameter. This parameter can be seen as a tool to control the complexity of the model, and to prevent it from adapting to noise in the training data.

In the SVM formulation given in the beginning of this section, it was stated that each data point should be on the correct side of the two planes that make up the margin. Mathematically, this was formulated as the constraint in equation (12).

Due to this constraint, the original formulation is very sensitive to noise in the training data. With linear SVM, a single noise point can lead to a margin of drastically reduced size, or an optimization problem that is not solvable at all.

In the case of non-linear SVM (using kernels), the constraint, in combination with noise in the training data, often leads to a model of unnecessary complexity. The decision boundary of such a model will curve and adapt to every single noise point in the data, resulting in problems of overfitting.

In [15], what is now called ”soft-margin” SVM was presented. With this modication, the primal optimization problem can be expressed as follows:

$$\min_{\omega,b} \frac{1}{2}\|\boldsymbol{\omega}\|^2 + C\sum_{i=1}^{N}\xi_i$$
$$\textit{subject to} \quad y_i(\boldsymbol{\omega}\cdot\boldsymbol{x_i}+b) \geq 1-\xi_i$$
$$i = 1,\ldots,N.$$

As can be seen, the constraint from (12) has been slightly modied. The soft-margin formulation of SVM introduces slack variables $_i$ that permits data

points to exist on the wrong side of the margin. By specifying the regularization parameter C, the user can decide how much ”slack” is allowed. The resulting SVM model is much more resistant to noise, since individual points are allowed to exist on the wrong side of the margin.

In the dual formulation of SVM, the soft margin implementation only leads to an additional constraint $\alpha_i < C$.

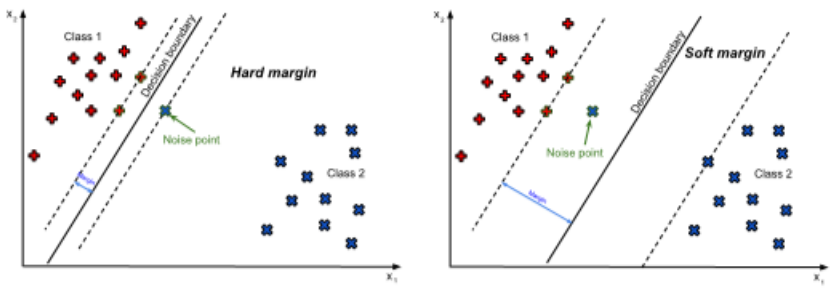


Figure A2: Illustration of soft-margin SVM

In figure above the benefit of soft margin is clear; a much wider margin is created instead of adapting the decision boundary to the individual noise point.

Model selection The predictive performance of a support vector machine model is dependent on not only the training data from which it is constructed, but on SVM parameters as well.

A normal scenario is that there are two parameters to select: the soft-margin cost parameter C, and a kernel parameter in the case of kernel-SVM.

When $C \to \infty$, the cost for slack in the model will be so high that it will in eect be a hard margin classier. This means that each training observation must be placed on the correct side of the decision boundary.

This will yield a model that always gets a 100% score on training data, but is extremely sensitive to overfitting and noise. As C gets smaller however, the error rate of the model will increase as more and more points will be placed on the wrong side of the decision boundary, and the model complexity is reduced.

The mathematical formulation of SVM, and the fact that this parameter can take any value, makes it hard to intuitively decide a suitable value for C, it is instead recommended to use some form of iterative method and choose the value which yields the best results for the dataset in question.

Appendix B : K-fold cross-validation Algorithm

Algorithm : K-fold cross validation process

Input : All data

Output: Score of the model

1 Shuffle the dataset randomly.
2 Split the dataset into k groups
3 **for** *each unique group* **do**
4 Take the group as a hold out or test data set
5 Take the remaining groups as a training data set
6 Fit a model on the training set and evaluate it on the test set
7 Retain the evaluation score and discard the model
8 **end**
9 Summarize the skill of the model using the sample of model evaluation scores

Appendix C : Principal Component Analysis Algorithm

Algorithm The PCA algorithm

1: **Input**: a D-dimensional training set $X = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N\}$ and the new (lower) dimensionality d (with $d \leq D$)

2: Compute the mean $\bar{\boldsymbol{x}} = \frac{1}{N} \sum_{i=1}^{N} \boldsymbol{x}_i$

3: Compute the covariance matrix $\text{Cov}(\boldsymbol{x}) = \frac{1}{N} \sum_{i=1}^{N} (\boldsymbol{x}_i - \bar{\boldsymbol{x}}_i) (\boldsymbol{x}_i - \bar{\boldsymbol{x}}_i)^T$

4: Find the spectral decomposition of $\text{Cov}(\boldsymbol{x})$, obtaining the eigenvectors $\boldsymbol{\xi}_1, \boldsymbol{\xi}_2, \ldots, \boldsymbol{\xi}_D$ and their corresponding eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_D$. Note that the eigenvalues are sorted, such that $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_D \geq 0$

5: For any $\boldsymbol{x} \in \mathbb{R}^D$, its new lower dimensional representation is:

$$\boldsymbol{y} = \left(\boldsymbol{\xi}_1^T(\boldsymbol{x} - \bar{\boldsymbol{x}}), \boldsymbol{\xi}_2^T(\boldsymbol{x} - \bar{\boldsymbol{x}}), \ldots, \boldsymbol{\xi}_d^T(\boldsymbol{x} - \bar{\boldsymbol{x}})\right)^T \in \mathbb{R}^d,$$

and the original $\boldsymbol{x}$ can be approximated as

$$\boldsymbol{x} \approx \bar{\boldsymbol{x}} + (\boldsymbol{\xi}_1^T(\boldsymbol{x} - \bar{\boldsymbol{x}}))\boldsymbol{\xi}_1 + (\boldsymbol{\xi}_2^T(\boldsymbol{x} - \bar{\boldsymbol{x}}))\boldsymbol{\xi}_2 + \cdots + (\boldsymbol{\xi}_d^T(\boldsymbol{x} - \bar{\boldsymbol{x}}))\boldsymbol{\xi}_d$$

Appendix D : Recursive Feature Elimination Algorithm

Algorithm : Recursive feature elimination

Tune/train the model on the training set using all predictors
Calculate model performance
Calculate variable importance or rankings
for *Each subset size* S_i, $i = 1 \ldots S$ **do**
    Keep the S_i most important variables
    [Optional] Pre–process the data
    Tune/train the model on the training set using S_i predictors
    Calculate model performance
    [Optional] Recalculate the rankings for each predictor
end
Calculate the performance profile over the S_i
Determine the appropriate number of predictors
Use the model corresponding to the optimal S_i

Bibliography

[1] "Wevioo consulting." https://www.wevioo.com/fr.

[2] "Ebsys groups." http://www.ebsys-tech.com/.

[3] "Selectkbest." https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.SelectKBest.html.

[4] "Extratreesclassifier." https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.ExtraTreesClassifier.html.

[5] "scikit-learn/principal component analysis." https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html.

[6] "Knn." https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.GaussianNB.html.

[7] "Knn." https://scikit-learn.org/stable/modules/neighbors.html.

[8] "Bnb." https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.BernoulliNB.html.

[9] "Lor." https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html.

[10] "Lsvc." https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html.

[11] "Nusvc." https://scikit-learn.org/stable/modules/generated/sklearn.svm.NuSVC.html.

[12] "Dtc." https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html.

[13] V. Vapnik and A. Lerner, *Pattern recognition using generalized portrait method*. 1963.

[14] I. M. G. B. E. Boser and V. N. Vapnik, "A training algorithm for optimal margin classiers," in *Proceedings of the Fifth Annual Workshop on Computational Learning Theory*, 1992.

[15] C. Cortes and V. Vapnik, "Support-vector networks," in *Machine Learning*, p. 273–297, 1995.

[16] W. T. V. W. H. Press, S. A. Teukolsky and B. Flannery, *Numerical recipes*. PhD thesis, Cambridge University Press, 2007.

[17] "Machine learning concepts." https://www.expertsystem.com/machine-learningdefinition/.

[18] "Simulate support vector machine (svm)." https://en.proft.me/2014/04/22/how-simulate-support-vector-machine-svm-r/.

[19] A. F. Victor Nordenmark, *Radar-detection based classification of moving objects using machine learning methods*. 2015.

[20] "Classication report.." https://www.scikit-yb.org/en/latest/api/classifier/classification_report.html.

[21] "Radra fmcw." http://www.radartutorial.eu/02.basics/Radar%20%C3%A0%20onde%20continue%20modul%C3%A9e%20en%20fr%C3%A9quence.fr.html. Accessed march. 15, 2019.

[22] "Principal component analysis." https://towardsdatascience.com/dive-into-pca-principal-component-analysis-with-python-43ded13ead21.

[23] Y. H. Xinyu Li and X. Jing, *A Survey of Deep Learning-Based Human Activity Recognition in Radar*. PhD thesis, School of Information and Communication Engineering, Beijing University of Posts and Telecommunications, Beijing 100876, China.

[24] A. R. D. G. t. A. M. K. A w miller, C Clemente and J. Soraghan, *Micro-Doppler based target classification using multi-feature integration*. PhD thesis, University of Strathclyde, Glasgow, UK, 2014.

[25] E. G. Menendez, *Frequency-modulated continuous-wave radar in automotive applications*. PhD thesis, Universitat Autonoma de Barcelona, 2018.

[26] J.-E. L. S.-C. K. Seongwook Lee, Young-Jun Yoon, *Human–vehicle classification using featurebased SVM in 77-GHz automotive FMCW radar*. PhD thesis, Department of Electrical and Computer Engineering and Institute of New Media and Communications (INMC), Seoul National University, Seoul, Republic of Korea, 2014.

[27] V. Micka, *Radar Target Classification by Micro-Doppler Contributions*. PhD thesis, Czech Technical University in Prague Faculty of Electrical Engineering Department of Cybernetics, 2017.

[28] Y. Kim and H. Ling, "Human activity classification based on micro-doppler signatures using a support vector machine," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 47, pp. 654–660, 2009.